Informática II

Carpeta de Trabajos Prácticos

Curso: 2R3 2012

Docentes:

- **Teórico:** Profesor Dr. Eduardo Destefanis.
- **Practico:** Profesora Ing. Silvia Carrera.

Alumnos:

*Da Silva, Luis Miguel	60099
*Duffaut, Rodrigo José	60454
*Rosso, Gabriel Alejandro	59871
*Sosa Morales, Saul Alberto	60711

Índice

T.P Nº 1: Repaso Informática I ----------P 3
T.P Nº 2: Punteros y arreglos---------- P 19
T.P Nº 3: Métodos Recursivos----------P 29
T.P Nº 5: Arreglos Dinámicos----------P 32
T.P Nº 6: Arreglos: Pilas y Colas----------P 44
T.P Nº 7: Listas Simples y Dobles ----------P 58
T.P Nº 8: Arboles Binarios ---------- P 91
T.P Nº 9: Programación Or. Objetos----------P 113
Anexos: sobrecarga de operadores ----------P 148

Trabajo Práctico Nº 1

Tema: Repaso Informática I

Objetivos:

*Repasar los contenidos desarrollados en el programa de la materia Informática I.

* Este repaso implica declaraciones de tipos permitidos, inicializaciones, tipos de estructuras de control, arreglos estáticos, métodos.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Ejercicio Nº 1:

```
#include <stdio.h>

void leer(float *);
void compara(float *);
int muestra(float *);

int main(int argc, char **argv)
{
      float nro[2];
      int bandera=1;
      printf(" division entre mayor y menor numero ingresados por teclado\n\n");
do
         {
      leer(nro);
      compara(nro);
      bandera=muestra(nro);
    }while(bandera==0);
      return 0;
      }

void leer(float *q){
    int i;
    for(i=0;i<2;i++){
                    printf("Ingrese el %d° nro:",i+1);
                    scanf("%f",(q+i));
                    }
    }

void compara(float *q){
    float aux;
    if(*q<*(q+1))
            {aux=*q;
            *q=*(q+1);
            *(q+1)=aux;
            }
    }

int muestra(float *q){
                if(*(q+1)!=0)
                        { printf("\n %.2f / %.2f = %.2f",*q,*(q+1),(*q)/(*(q+1)));
```

```
                                                    return 1;
                                }
else
{printf("\nno se puede dividir por cero, ingrese los nuevos numeros\n");
return 0;}
                }
```

Ejercicio Nº 2:

```
#include <stdio.h>
int i;
void leer(float []);
void sumar(float ,float );
void compara(float []);
int main(int argc, char **argv)
{
      float n[3];
      printf(" suma del mayor y el menor numero\n");
      leer(n);
      compara(n);
      return 0;
      }
void compara(float a[]){
                  float mayor=a[0];
                  float menor=a[0];
                  for(i=1;i<3;i++){
                              if(mayor<a[i])
                                          mayor=a[i];
                              if(menor>a[i])
                                          menor=a[i];
                              }
                  sumar(menor,mayor);
                  }

void sumar(float n1,float n2){
                   float res=n1+n2;
                   printf("\n %.2f + %.2f = %.2f\n",n1,n2,res);
                   }
void leer(float a[])
{

         for(i=0;i<3;i++)
         {
```

```
                printf(" Ingrese el %d nro:",i+1);
                scanf("%f",&a[i]);
                if(i!=0)
                        {
                                if(a[i]==a[i-1]||a[i]==a[i-2])
                                {printf("el numero ingresado es igual a un numero ya ingresado,
ingreselo nuevamente\n");
                                i--;
            }
                        }
        }
}
```

Ejercicio Nº 3:

```
#include <stdio.h>

int main(int argc, char **argv)
{int i,a[]={2,-3,30,-1,-1,50,22,100,0,5};
                printf("este programa muestra los numeros positivos de un arreglo con
datos ya cargados\n");
                for (i = 0; i < 10; i++)
                {       if(a[i]>0)
                                {printf("el numero positivo es:%d\t\n",a[i]);
                                }
                }
        return 0;
}
```

Ejercicio Nº 4:

```
#include <stdio.h>
void arreglo(int,int[]);
int main(int argc, char **argv)
{       int a[20],i,count=0;
        printf("este programa copia los numeros pares de un arreglo a otro\n");
        for (i = 0; i < 20; i++)
        {
                count ++;
                printf("ingrese el numero %d\n",i+1);
                scanf("%d",&a[i]);
        }
```

```
arreglo(count,a);

	return 0;
}

void arreglo (int c,int b[])
{ int arr[c],i,k=0;
	for (i = 0; i < c; i++)
	{	if((b[i]%2)==0)
			{
				arr[k]=b[i];
				k++;
			}

	}
	for (i = 0; i < k; i++)
	{	printf(" %d° numero par del arreglo copiado es=\t%d\n",i+1,arr[i]);
	}
}
```

Ejercicio Nº 5:

```
#include <stdio.h>
void arreglo(int,int[]);
int main(int argc, char **argv)
{	int a[20],i,count=0;
	printf("este programa copia los numeros pares de un arreglo a otro\n");
	for (i = 0; i < 20; i++)
	{	count++;
		printf("ingrese el numero %d\n",i+1);
		scanf("%d",&a[i]);
	}


arreglo(count,a);

	return 0;
}
```

```
void arreglo (int c,int b[])
{ int arr[c],i,k=0;
        for (i = 0; i < c; i++)
                {
                        if(b[i]!=b[i+1])
                                {       arr[k]=b[i];
                                        k++;
                                }
                }

        for (i = 0; i < k; i++)
        {       printf(" %d° numero del arreglo copiado donde no se muestran
repeticiones=\t%d\n",i+1,arr[i]);
        }
}
```

Ejercicio N° 6:

```
#include <stdio.h>
void copy(int, char[]);
int main(int argc, char **argv)
{       char a[15];
        int i,count=0;
        printf("este programa copia las mayusculas en otro arreglo\n");
        for (i = 0; i < 15; i++)
        {       printf("ingrese las letras en mayuscula y minuscula\n");
                scanf("%s",&a[i]);
        }
for (i = 0; i < 15; i++)
{
        if (a[i]<97)
        {
                count++;
        }
copy(count,a);

}



        return 0;
}

void copy(int b, char c[])
```

```
{       char d[b];
        int i=0,j=0;
        do
        {
                        if(c[i]<97)
                        {       d[j]=c[i];
                                i++;
                                j++;
                        }
                        else
                        {       i++;
                        }
                }while(i!=b);
        for (i = 0; i < b; i++)
                {
                        printf("caracter solo en minuscula:\t %c \n", d[i]);
                }
}
```

Ejercicio Nº 7:

```
#include <stdio.h>
#define tam 5
void fun1(char *);
int mayus(char *);
int minus(char *);
int verificar(char *);
void copiar(char *,char *);
int recorrer(char *,char *);
void imprime(char *,int);
int main(int argc, char **argv)
{
      char arr1[tam],arr2[tam];
      int res;
      printf(" el programa copia las letras de un arreglo a otro\n");
      fun1(arr1);
      res=recorrer(arr1,arr2);
      printf("\nArreglo 1:\n");
      imprime(arr1,tam);
      printf("\nArreglo 2:\n");
      imprime(arr2,res);
      return 0;
      }
```

```
void fun1(char *A){
        int i;
            for(i=0;i<tam;i++){
                        printf("1[%d]=",i+1);
                        scanf("%s",A+i);
                        }
            }

int mayus(char *c){
            if((*c<91)&&(64<*c))
                        return 1;
            else
               return 0;
            }
int minus(char *c){
            if((*c<123)&&(96<*c))
                        return 1;
            else
               return 0;
            }
int verificar(char *caracter){
                    if((mayus(caracter))||(minus(caracter)))
                                                return 1;
                    else
                       return 0;
                    }
void copiar(char *c1,char *c2){
                     *c2=*c1;
                     }
int recorrer(char *A,char *B){
                    int total=0,i;
                    for( i=0;i<tam;i++)
                                if(verificar(A+i)==1){
                                            copiar(A+i,B+total);
                                            total++;
                                            }
                    return total;
                    }
void imprime(char *vec,int cant){
        int i;
                       for(i=0;i<cant;i++)
```

```
                                        printf("   %c\n",*(vec+i));
                        }
```

Ejercicio Nº 8:

```
#include <stdio.h>
#define tam 3
int main(int argc, char **argv)
{int m[tam][tam],i,j;
printf("este programa imprime la diagonal principal de una matriz 3x3\n");
for (i = 0; i < tam; i++)
{       for(j=0;j<tam;j++)
                {       printf("\nA[%d][%d]=\t",i+1,j+1);
                        scanf("%d",&m[i][j]);

                }
}
printf("\n\ndiagonal principal\n\n");
for (i = 0; i < tam; i++)
{       printf("%d",m[i][i]);
                if(i==0)
                {printf("\t\t\n\t");}
                if(i==1)
                {printf("\t\n\t\t");}
                if(j==2)
                {printf("\n");}

}
        return 0;
}
```

Ejercicio Nº 9:

```
#include <stdio.h>
#define tam 3
int main(int argc, char **argv)
{int m[tam][tam],i,j;
printf("este programa imprime la diagonal principal de una matriz 3x3\n");
for (i = 0; i < tam; i++)
{       for(j=0;j<tam;j++)
                {       printf("\nA[%d][%d]=\t",i+1,j+1);
                        scanf("%d",&m[i][j]);
```

```
                }
}
printf("\n\ndiagonal secundaria\n\n");
for (i = 0; i < tam; i++)
{                       if(i==0)
                        {printf("\t\t");
                        printf("%d",m[i][2]);}
                        if(i==1)
                        {printf("\n\t");
                        printf("%d",m[i][i]);}
                        if(i==2)
                        {printf("\t\n");
                        printf("%d",m[i][0]);}
}




        return 0;
}
```

Ejercicio Nº 10:

```
#include <stdio.h>
#define tam 4
int main(int argc, char **argv)
{int m[tam][tam],i,j,may,men;
printf("este programa imprime la diagonal principal de una matriz 3x3\n");
for (i = 0; i < tam; i++)
{       for(j=0;j<tam;j++)
                {       printf("\nA[%d][%d]=\t",i+1,j+1);
                        scanf("%d",&m[i][j]);

                }
}
may=men=m[1][1];
for (i = 0; i < tam; i++)
{       for (j = 0; j < tam; j++)
                {       if (may<m[i][j])
                                {
                                        may=m[i][j];
```

```
                        }
                    if (men>m[i][j])
                        {
                            men=m[i][j];
                        }

                }

}
printf("\tmayor=%d\n\tmenor=%d",may,men);


        return 0;
}
```

Ejercicio Nº 12:

```
#include <stdio.h>
#define tam 3

void ingresa(char *,char *);
void compara(char *,char *,int );
void recorrer(char *,char *);
void mostrar(char *);
int i,j;
int main(int argc, char **argv)
{
    char mat1[tam][tam],mat2[tam][tam];
    printf(" ejercicio 12_guia1\n\n");
    ingresa(mat1[0],mat2[0]);
    recorrer(mat1[0],mat2[0]);
    printf("\n\n matriz 1:\n");
    mostrar(mat1[0]);
    printf("\n\n matriz 2:\n");
    mostrar(mat2[0]);
    return 0;
    }

void ingresa(char *m1,char *m2)
{
        for(i=0;i<tam;i++)
            {for(j=0;j<tam;j++)
                {
```

```
                        printf("A[%d][%d]=",i+1,j+1);
                        scanf("%s",m1+i*tam+j);
                        *(m2+i*tam+j)=*(m1+i*tam+j);
        }
    }
}

void compara(char *m1,char *m2,int n)
{
    for(i=n+1;i<tam*tam;i++)
        {
            if(*(m1+i)==*(m2+n))
            {
                *(m2+i)=' ';
            }
        }
}

void recorrer(char *m1,char *m2)
{
    for(i=0;i<tam;i++)
        {
            for(j=0;j<tam;j++)
                {if(*(m2+i*tam+j)!=' ')
                    {compara(m1,m2,i*tam+j);
                    }
                }
        }
}
void mostrar(char *matriz)
{
    for(i=0;i<tam;i++)
        {
            printf("\t");
            for(j=0;j<tam;j++)
                {
                    printf("%c\t",*(matriz+i*tam+j));
                    printf("\n");
        }
    }
}
```

Ejercicio Nº 13 – 14:

```
#include <stdio.h>
#define tam 3
int main(int argc, char **argv)
{       int m[tam][tam]={{2,3,1},{4,5,1},{50,2,4}};
        int i,res[tam];
printf("ej13y14 guia_1\n");
for (i = 0; i < tam; i++)
{
                res[i]=m[i][0]+m[i][1]+m[i][2];
}
for (i = 0; i < tam; i++)
{
        printf("resultado de la suma de la fila %d=%d\n",i+1,res[i]);
}


        return 0;
}
```

Ejercicio Nº 15:

```
#include <stdio.h>
#define tam1 3
#define tam 5
void ingresa(int *);
void muestra(int *);
void aplazados(int *);
void prom(int *);
void aprobados(int *);
void prom_t(int *);
void operacion(char opc,int *m);
int i,j;
int main(int argc, char **argv)
{
    int matriz[tam][tam1],op;
        printf("ej15_guia1\n");
    ingresa(matriz[0]);
    muestra(matriz[0]);
    do
```

```
    {
                printf("\n\n  MENU:\n1- Numero de aplazados\n2- Pormedio por
materia\n3- Numero de aprobados\n4- Promedio notas\n5- Salir\n\n");
                scanf("%d",&op);
                operacion(op,matriz[0]);
            }while(op!=5);
    return 0;
}
void ingresa(int *m)
{
        for(i=0;i<tam;i++)
            {
                printf("\n\nAlumno %d:\n",i+1);
                    for(j=0;j<tam1;j++)
                        {
                            do
                            {
                                printf("Nota %d:",j+1);
                                scanf("%d",m+i*tam+j);
                            }while((*(m+i*tam+j)<1)||(*(m+i*tam+j)>10));
                        }
            }
}

void muestra(int *m)
{
        for(i=0;i<tam;i++)
            {
                    printf("\t");
                for(j=0;j<tam1;j++)
                    {
                        printf("%d",*(m+i*tam+j));
                        if (j==2)
                        {printf("\n");}
                        else
                        {printf("\t");}
                    }
            }
}

void aplazados(int *m)
{
        int total=0;
```

```
		for(i=0;i<tam;i++)
			{
				for(j=0;j<tam1;j++)
					{
						if(*(m+i*tam+j)<4)
							{total++;}
					}
			}
printf("\n las notas menores a 4 son=%d\n",total);
}

void prom(int *m)
{
	printf("El promedio por materia es:\n");
	float prom[tam1]={0};
	for(j=0;j<tam1;j++)
		{
			for(i=0;i<tam;i++)
			{
			prom[j]+=*(m+i*tam1+j);
			}
		}
printf("\nMateria %d: %.2f",j+1,prom[j]/tam);
}


void aprobados(int *m)
{
	int total=0;
	for(i=0;i<tam;i++)
		{
			for(j=0;j<tam1;j++)
				{
					if(*(m+i*tam+j)>=4)
					total++;
				}
		}
printf("\n numero de notas aprobadas=%d\n",total);
}

void prom_t(int *m)
{
	float prom=0;
```

```
        for(i=0;i<tam;i++)
                {
                        for(j=0;j<tam1;j++)
                                {
                                        prom+=*(m+i*tam+j);
                                        printf("\n El promedio total es de %.2f\n",prom/
(tam*tam1));
                                }
                }
}

void operacion(char opc,int *m)
{
        switch(opc){
                                case 1:
                aplazados(m);
                break;
            case 2:
                prom(m);
                break;
            case 3:
                aprobados(m);
                break;
            case 4:
                prom_t(m);
                break;
            default:
                printf("\n\n fin");
            }
}
```

Trabajo Práctico Nº 2

Temas: Punteros. Arreglos dinámicos. Relación entre punteros y arreglos. Arreglos de punteros. Métodos: procedimientos y funciones, invocación de métodos desde otra función. Puntero a funciones. Métodos como parámetros. Argumentos predeterminados

Objetivos:

- *Aprender a crear y utilizar arreglos dinámicos.
- *Comprender la vinculación que existe entre el nombre o identificador de un arreglo y los punteros
- *Aprender a construir programas de manera modular,a partir de piezas pequeñas llamadas métodos.
- *Entender los mecanismos que tienen el lenguaje para pasar información entre funciones.
- *Comprender la diferencia que existe entre los procedimientos y funciones, y su implementación en C.
- *Aprender a usar argumentos.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Ejercicio Nº 1:

```
#include <stdio.h>
#include <malloc.h>
#define tam 10
int condicion(int *c);
int main(int argc, char **argv)
{
		int *p=(int *)malloc(tam*sizeof(int));
		int i;
		printf(" creacion de un arreglo dinamico de tamaño %d y impresion de los
multiplos del tamaño\n\n",tam);
		for(i=0;i<tam;i++)
			{
				printf("A[%d]=",i+1);
				scanf("%d",p+i);
			}
		for(i=0;i<tam;i++)
			{	if(condicion(p+i)==1)
				printf("\n%d",*(p+i));
			}
return 0;
}
int condicion(int *var)
{
	if(*var%tam==0)
	return 1;
	else
	return 0;
}
```

Ejercicio Nº 2:

```
#include<stdio.h>
#include<stdlib.h>
#define tam 100

void cargar (int *,int *);
void cuenta(int *,int *, int *);
```

```
void imprime(int *,int *);
void menu(void);


int main (void)
{
int vector[tam]={0}, positivos=0, tamanio=0, select=0;
        while(select!=4)
        {
                menu();
                scanf("%d",&select);
                switch(select)
                {
                        case 1: system("clear");
                                cargar(vector,&tamanio);
                                system("clear");
                                        break;
                        case 2: system("clear");
                                imprime(vector,&tamanio);
                                        break;
                        case 3: system("clear");
                                cuenta(vector,&tamanio,&positivos);
                                        printf("La cantidad de numeros positivos en el arreglo es:
%d\n",positivos);
                                        break;
                  case 4: select = 4;
                        break;
                }
  }
return(0);
}

void menu(void)
{
        printf("\n1-Cargar el arreglo\n");
        printf("\n2-Imprimir el arreglo\n");
        printf("\n3-Contar los positivos el arreglo\n");
        printf("\n4-Salir");
}

void cargar(int *p, int *tama)
{
        int i;
```

```
        printf("Ingrese el tamanio del arreglo\n\n");
        scanf("%d",&(*tama));
        for(i=0;i<(*tama);i++)
        {
                printf("Ingrese el elemento %d\n",i+1);
                scanf("%d",&*(p+i));
    }
}

void imprime(int *p, int *tama)
{
        printf("Impresion del array\n\n");
        int i;
        for(i=0;i<(*tama);i++)
        {
                printf(" %d ",*(p+i));
        }
}

void cuenta (int *p, int *tama, int *positi)
{
        int i;
   for(i=0;i<(*tama);i++)
   {
                if(*(p+i)>0)
                (*positi)++;
                ;
        }
}
```

Ejercicio Nº 3:

```
#include<stdio.h>
#include<stdlib.h>
#define tam 100

void cargar(int *, int *);
void eliminarep(int *, int *);
void imprime(int *, int *);

int main(void)
{
```

```
        int vector[tam], tamanio;
        cargar(vector,&tamanio);
        printf("Vector con posibles repeticiones\n\n");
        imprime(vector,&tamanio);
        eliminarep(vector,&tamanio);
        printf("\nVector sin repeticiones\n\n");
        imprime(vector,&tamanio);
return(0);
}

void cargar(int *p, int *tamanio)
{ int i;

  printf("Ingrese el tamanio del arreglo (menor a 100)\n");
  scanf("%d",&(*tamanio));
  for(i=0;i<(*tamanio);i++)
   {
        printf("Ingrese el elemento %d\n",i+1);
        scanf("%d",&*(p+i));
    }

}

void imprime(int *p, int *tamanio)
{ int i;
  printf("TAMANIO %d\n\n",(*tamanio));
  for(i=0;i<(*tamanio);i++)
   {
        printf(" %d ",*(p+i));
   }

}

void eliminarep(int *p, int *tamanio)
{ int i,nt=0,flag=0,j=0;
  int nuevo[tam];
  for(i=0;i<(*tamanio);i++)
   { j=0;
        while((j<=nt))
         {
              if(*(p+i)==nuevo[j])
               flag=1;
               j++;
```

```
        }
        if(flag!=1)
        {
                nuevo[nt]=*(p+i);
                nt++;
          flag=0;
        }
   }
*tamanio=nt;
p=nuevo;
}
```

Ejercicio Nº 4:

```
#include<stdio.h>
#include<stdlib.h>

#define t 100;

void menu(void);
void carga(int *,int *);
void muestra (int *, int*);
void eliminarep(int *, int *);
void busca (int *, int *);


int main (void)
{
        int vector[100];
        int tam=0, select=0;
        while(select!=5)
        {
                menu();
                scanf("%d",&select);
                switch(select)
                {
                        case 1: carga(vector,&tam);
                                        system("clear");
                              break;
                        case 2: system("clear");
                              muestra(vector,&tam);
                              break;
```

```
                        case 3: system("clear");
                                busca(vector,&tam);
                                break;
                        case 4: select=4;
                                break;
          }
        }
return(0);
}

void menu (void)
{
        printf("1-Cargar datos al arreglo\n\n");
        printf("2-Mostrar el arreglo\n\n");
        printf("3-Buscar un valor en el arreglo\n\n");
        printf("5-Salir \n\n");
}

void carga (int *p, int *tam)
{
        int i;
        printf("Ingrese el tamanio del arreglo (debe ser menor a 100)\n");
        scanf("%d",&(*tam));
        for(i=0;i<*tam;i++)
        {
                printf("Ingrese el elemento %d\n\n",i+1);
                scanf("%d",&*(p+i));
        }
}

void muestra (int *p, int *tam)
{
        printf("Imprimimos el arreglo de tamanio %d\n\n",(*tam));
        int i;
                for(i=0;i<*(tam);i++)
                printf("%d\n\n",*(p+i));
}

void busca (int *p, int *tam)
{
        int valor, flag=0, posicion=0,i=0;
        if((*tam)==0)
        {
```

```
            printf("El arreglo esta vacio");
        }
        else
        {
            printf("Ingrese un valor a buscar\n");
            scanf("%d",&valor);
            while((flag==0)&&(i<(*tam)))
            {
                if(valor==(*(p+i)))
                 {
                    flag=1;
                    posicion=i;
                }
                else
                i++;
            }
            if(flag==1)
          printf("El valor esta en la posicion %d\n\n",posicion+1);
          else
          printf("El valor NO esta\n\n");
   }
}
```

Ejercicio Nº 5:

```
#include <stdio.h>
#include <malloc.h>
#define tam 5
int main(int argc, char **argv)
{       int *vec=(int *)malloc(tam*sizeof(int));
        int i,j,aux;
        printf("este programa ordena elementos de un arreglo y los muestra\n\n ingrese
datos\n");
        for (i = 0; i < tam; i++)
        {       printf("vec[%d]=",i);
                scanf("%d",&vec[i]);
```

```
	}
	for(i=0;i<tam;i++)
		{for(j=0;j<(tam-i);j++)
			if(vec[j]>vec[j+1])
				{	aux=vec[j];
					vec[j]=vec[j+1];
					vec[j+1]=aux;
				}
		}
	printf("vector ordenado\n\n");
	for (i = 0; i < tam; i++)
	{	printf("vec[%d]=%d\n",i,vec[i]);
	}
	return 0;
}
```

Ejercicio Nº 6:

-

```
#include <stdio.h>
#include <malloc.h>
#define tam 3
int main(int argc, char **argv)
{int m[tam][tam],i,j;
printf("este programa imprime la diagonal principal de una matriz 3x3\n");
for (i = 0; i < tam; i++)
{	for(j=0;j<tam;j++)
		{	printf("\nA[%d][%d]=\t",i+1,j+1);
			scanf("%d",&m[i][j]);

		}
}
     int *vec=(int *)malloc(tam*sizeof(int));
for (i = 0; i < tam; i++)
{
	vec[i]=m[0][i];

}
printf("fila1\n");
for (i = 0; i < tam; i++)
{
	printf("vec[%d]=%d\n",i+1, vec[i]);
}
```

```
	return 0;
}
```

Ejercicio Nº 7:

```
#include<stdio.h>
#include<stdlib.h>

void carga (int *);
void muestra (int **);

int main(void)
{
int *p,**g;
carga(p);
**g=&p;
printf("Imprime desde main %d\n\n",*p);
muestra(g);
return(0);
}

void carga(int *p)
{
int dato;
printf("Dato: ??\n\n");
scanf("%d",&dato);
*p=dato;
}

void muestra(int **g)
{
	printf("entra a muestra");
	int dato=**g;
printf("Dato: %d",dato);
}
```

Trabajo Práctico Nº 3

Tema: Métodos Recursivos.

Objetivos:

*Aprender a implementar métodos recursivos, comprender las diferencias que existen entre los métodos recursivos y no recursivos.

* Discriminar las principales partes que tienen los métodos recursivos.

*Establecer criterios y parámetros de análisis que permitan comparar métodos recursivos y no recursivos.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Ejercicio Nº 1:

```
#include<stdlib.h>

int recu (int,int);

int main(void)
{
        int base,res,exp;
        printf("Ingrese la base\n\n");
        scanf("%d",&base);
        printf("Ingrese el exponente\n\n");
        scanf("%d",&exp);
        res = recu(base,exp);
   printf("El resultado es %d",res);
return(0);
}

int recu(int base, int exp)
{
        if(exp==0)
        return(1);
        else
        return(base*(recu(base,exp-1)));
}
```

Ejercicio Nº 2:

```
#include<stdio.h>
#include<stdlib.h>

int facto (int);

int main(void)
{
        int base,res;
        printf("Ingrese la base\n\n");
        scanf("%d",&base);
        res = facto(base);
   printf("El resultado es %d",res);
return(0);
}
```

```
int facto(int base)
{
        if(base==0)
        return(1);
        else
        return(base*facto(base-1));
}
```

Ejercicio Nº 4:

```
#include <stdio.h>
int recu(int n);
int main()
{
   int dato,res;
   printf("ingrese el numero:\n");
   scanf("%d",&dato);
   res=recu(dato);
   printf("el resultado es:%d\n",rdo);
   return 0;

}
int recu(int n)
{
   if(n==1)
   {
      return 1;
   }
   else
   {
     return recu(n-1)+2*n-1;
   }
}
```

Trabajo Práctico Nº 5

Tema: Uso del tipo de dato struct. Arreglos estáticos y dinámicos de tipo struct. Tipo de dato struct como parámetro formales y actuales. Métodos que devuelven struct.

Objetivos:

- *Aprender a declarar, inicializar y crear variables de tipo struct.
- *Comprender la diferencia entre arreglos y tipos de dato struct.
- *Comprender la forma en que el tipo de dato struct pasa como argumento de un método.
- *Aprender a realizar estructuras anidadas.
- * Aprender a inicializar estructuras.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Ejercicio Nº 1:

```
#include <stdio.h>
#include <malloc.h>
#define TAM 5
struct valores{
	int num1;
	int num2;
	float res;
	};


int main(int argc, char **argv)
{	int i;
	struct valores *p;
	p=malloc(TAM*sizeof(struct valores));
	for (i = 0; i < TAM; i++)
	{
		printf("Ingrese el 1er valor de la estructura %d:\n",i+1);
		scanf("%d",&(*(p+i)).num1);
		printf("Ingrese el 2do valor de la estructura %d:\n",i+1);
		scanf("%d",&(*(p+i)).num2);
	}
	for (i = 0; i < TAM; i++)
	{
		(*(p+i)).res=((*(p+i)).num1+(*(p+i)).num2)/2;
	}
	for (i = 0; i < TAM; i++)
	{
		printf("Estructura %d: N1: %d N2: %d\nPromedio:%.2f\n",i+1,(*(p+i)).num1,
(*(p+i)).num2,(*(p+i)).res);
	}


	return 0;
}
```

Ejercicio Nº 2:

```
#include <iostream>
using namespace std;
#define tam 10
```

```
# include <malloc.h>

struct ejerc
{
	float temp_max,temp_min;
	int enc;
};

int main()
{
	struct ejerc elem[tam];
	int i,cont[20],cant=0;
	float *p;

	for(i=0;i<tam;i++)
	{
		cout<<"Ingrese la temperatura maxima "<<i+1<<": ";
		cin>>elem[i].temp_max;
		cout<<"Ingrese la temperatura minima "<<i+1<<": ";
		cin>>elem[i].temp_min;
		cout<<"Ingrese numero de veces de encendido "<<i+1<<": ";
		cin>>elem[i].enc;
		cout<<"\n";

		if( elem[i].enc < 3 )
		{		cont[i] = i; cant=cant+1;  }
	}

	p = new (float[cant]);

	for(i=0;i<cant;i++)
		*(p+i) = elem[ cont[i] ].temp_max;

	cout<<"\n\nLa estructura original cargada es: "<<endl;
	for(i=0;i<tam;i++)
	{
		cout<<"Temperatura maxima"<<i+1<<": "<<elem[i].temp_max<<endl;
		cout<<"Temperatura minima"<<i+1<<": "<<elem[i].temp_min<<endl;
		cout<<"Cantidad de encendidos"<<i+1<<": "<<elem[i].enc<<"\n\n";
	}

	cout<<"\n\nLa estructura con menos de 3 encendidos es: "<<"\n";
	for(i=0;i<cant;i++)
```

```
	{
		cout<<"Encendidos"<<i+1<<" :"<<*(p+i)<<endl;
	}

	return 0;
}
```

Ejercicio Nº 2.2:

```
#include <iostream>
using namespace std;
#define tam 10
# include <malloc.h>

struct ejerc
{
	float temp_max,temp_min;
	int enc;
};

int main()
{
	struct ejerc elem[tam];
	int i,cont[20],cant=0;
	float *p;

	for(i=0;i<tam;i++)
	{
		cout<<"Ingrese la temperatura maxima "<<i+1<<": ";
		cin>>elem[i].temp_max;
		cout<<"Ingrese la temperatura minima "<<i+1<<": ";
		cin>>elem[i].temp_min;
		cout<<"Ingrese numero de veces de encendido "<<i+1<<": ";
		cin>>elem[i].enc;
		cout<<"\n";

		if( elem[i].enc < 3 )
		{		cont[i] = i; cant=cant+1;  }
	}

	p = new (float[cant]);

	for(i=0;i<cant;i++)
```

```
            *(p+i) = elem[ cont[i] ].temp_max;

        cout<<"\n\nLa estructura original cargada es: "<<endl;
        for(i=0;i<tam;i++)
        {
            cout<<"Temperatura maxima"<<i+1<<": "<<elem[i].temp_max<<endl;
            cout<<"Temperatura minima"<<i+1<<": "<<elem[i].temp_min<<endl;
            cout<<"Cantidad de encendidos"<<i+1<<": "<<elem[i].enc<<"\n\n";
        }

        cout<<"\n\nLa estructura con menos de 3 encendidos es: "<<"\n";
        for(i=0;i<cant;i++)
        {
            cout<<"Encendidos"<<i+1<<" :"<<*(p+i)<<endl;
        }

        return 0;
}
```

Ejercicio Nº 3,3:

```
#include <iostream>
using namespace std;
#define tam 10
# include <malloc.h>

struct ejerc
{
        float tiempo,presion;
        int num_ap;
};

int main()
{
        struct ejerc elem[tam];
        int i;
        float contador=0;

        for(i=0;i<tam;i++)
        {
            cout<<"Ingrese el tiempo "<<i+1<<": ";
            cin>>elem[i].tiempo;
```

```
		cout<<"Ingrese la presion de corte "<<i+1<<": ";
		cin>>elem[i].presion;
		cout<<"Ingrese numero de veces de encendido "<<i+1<<": ";
		cin>>elem[i].num_ap;
		cout<<"\n";
		contador=contador + elem[i].tiempo;
	}
	contador = contador/tam;


	cout<<"EL promedio del tiempo es:"<<contador;

	cout<<"\n\nLos ciclos que estuvieron por encima del promedio de tiempo son :
"<<"\n";
	for(i=0;i<tam;i++)
	{
		if( elem[i].tiempo > contador )
		{
			cout<<"Tiempo"<<i+1<<": "<<elem[i].tiempo<<endl;
			cout<<"Presion de corte"<<i+1<<": "<<elem[i].presion<<endl;
			cout<<"Numero de aperturas"<<i+1<<": "<<elem[i].num_ap<<"\n\n";
		}
	}

	return 0;
}
```

Ejercicio Nº 3:

```
#include <iostream>
using namespace std;
#define tam 10
# include <malloc.h>

struct ejerc
{
	float tiempo,presion;
	int num_ap;
};

int main()
{
	struct ejerc elem[tam];
	int i;
	float contador=0;
```

```
	for(i=0;i<tam;i++)
	{
		cout<<"Ingrese el tiempo "<<i+1<<": ";
		cin>>elem[i].tiempo;
		cout<<"Ingrese la presion de corte "<<i+1<<": ";
		cin>>elem[i].presion;
		cout<<"Ingrese numero de veces de encendido "<<i+1<<": ";
		cin>>elem[i].num_ap;
		cout<<"\n";
		contador=contador + elem[i].tiempo;
	}
	contador = contador/tam;


	cout<<"EL promedio del tiempo es:"<<contador;

	cout<<"\n\nLos ciclos que estuvieron por encima del promedio de tiempo son :
"<<"\n";
	for(i=0;i<tam;i++)
	{
		if( elem[i].tiempo > contador )
		{
			cout<<"Tiempo"<<i+1<<": "<<elem[i].tiempo<<endl;
			cout<<"Presion de corte"<<i+1<<": "<<elem[i].presion<<endl;
			cout<<"Numero de aperturas"<<i+1<<": "<<elem[i].num_ap<<"\n\n";
		}
	}

	return 0;
}
```

Ejercicio Nº 4.4:

```
#include <iostream>
using namespace std;
#include <stdlib.h>

struct suma
{
	float real,imag;
};
```

```
int main ()
{
	struct suma num1,num2;
	int band=0;

	while (band!=-1)
	{
		cout<<"Complejo 1"<<endl;
		cout<<"Real: ";cin>> num1.real;
		cout<<"Imaginario: ";cin>> num1.imag;

		cout<<"\nComplejo 2"<<endl;
		cout<<"Real: ";cin>> num2.real;
		cout<<"Imaginario: ";cin>> num2.imag;

		cout<<"\nLa suma es:";
		cout<<num1.real + num2.real<<",";
		cout<<num1.imag + num2.imag<<"i";

		cout<<"\nIngrese -1 para salir";
		cin>>band;
		system("clear");
	}

	return 0;
}
```

Ejercicio Nº 4:

```
#include <iostream>
using namespace std;
#include <stdlib.h>

struct suma
{
	float real,imag;
};


int main ()
{
	struct suma num1,num2;
	int band=0;
```

```
        while (band!=-1)
        {
                cout<<"Complejo 1"<<endl;
                cout<<"Real: ";cin>> num1.real;
                cout<<"Imaginario: ";cin>> num1.imag;

                cout<<"\nComplejo 2"<<endl;
                cout<<"Real: ";cin>> num2.real;
                cout<<"Imaginario: ";cin>> num2.imag;

                cout<<"\nLa suma es:";
                cout<<num1.real + num2.real<<",";
                cout<<num1.imag + num2.imag<<"i";

                cout<<"\nIngrese -1 para salir";
                cin>>band;
                system("clear");
        }

        return 0;
}
```

Ejercicio Nº 11:

```
#include <iostream>
using namespace std;
#define tam 5
#include <math.h>

struct punto
{
        int x,y;
};

int main()
{
        struct punto point[tam];
        float distancia;
        int i;

        for(i=0;i<tam;i++)
        {
                cout<<"Punto "<<i+1;
```

```
            cout<<"Ingrese la coordenada X: ";
            cin>>point[i].x;
            cout<<"Ingrese la coordenada Y: ";
            cin>>point[i].y;
            cout<<"\n";
        }

        for(i=0;i<tam;i++)
        {
            cout<<"punto "<<i+1<<": ("<<point[i].x<<","<<point[i].y<<")\n";

            if( point[i].x > 0 )
            {
                if(point[i].y > 0)
                {cout<<"El punto se encuentra en el primer cuadrante\n";}
                else
                {cout<<"El punto se encuentra en el cuarto cuadrante\n"; }
            }
            else
            {
                if(point[i].y > 0)
                { cout<<"El punto se encuentra en el segundo cuadrante\n"; }
                if(point[i].y < 0)
                { cout<<"El punto se encuentra en el tercer cuadrante\n"; }
            }
            if( point[i].x == 0 && point[i].y == 0 )
            cout<<"El punto se encuentra en el origen\n";
            else
            {
                if ( point[i].x == 0 )
                cout<<"Se encuentra en el eje de las X";
                if ( point[i].y == 0 )
                cout<<"Se encuentra en el eje de las Y";
            }

            distancia = sqrt(  ( (point[i].x*point[i].x)) +   (point[i].y*point[i].y)  );
            cout<<"La distancia al origen es: "<<distancia<<"\n\n";

        }

        return 0;
}
```

Ejercicio Nº 12:

```
#include <iostream>
using namespace std;

struct persona
{
	char nombre[30];
	int edad,altura,peso;
};

void mostrar( struct persona * );
void imp ( struct persona * );

int main()
{
	struct persona elem[3];
	struct persona *punt;
	int i;

	for (i=0;i<3;i++)
	{
		cout<<"Persona "<< i+1<<": "<<endl;
		cout<<"Ingrese el nombre: ";cin>>elem[i].nombre;
		cout<<"Ingrese la edad: ";cin>>elem[i].edad;
		cout<<"Ingrese la altura: ";cin>>elem[i].altura;
		cout<<"Ingrese el peso: ";cin>>elem[i].peso;cout<<"\n";
	}
	punt=elem;

	mostrar(punt);
	imp(punt);

	return 0;
}
void mostrar ( struct persona * puntero )
{
	int i;
	cout<<"\nLos datos ingresados son:\n";
	for(i=0;i<3;i++)
	{
		cout<<"Nombre: "<<(puntero+i)->nombre<<endl;
		cout<<"Edad: "<<(puntero+i)->edad<<endl;
		cout<<"Altura: "<<(puntero+i)->altura<<endl;
```

```
                cout<<"Peso: "<<(puntero+i)->peso<<endl;
                cout<<"\n";
        }
}
void imp ( struct persona * puntero )
{
        int i;
        cout<<"\nMayores de edad\n";
        for(i=0;i<3;i++)
        {
                if(  (puntero+i)->edad > 18  )
                {
                        cout<<"Nombre: "<<(puntero+i)->nombre<<endl;
                        cout<<"Edad: "<<(puntero+i)->edad<<endl;
                        cout<<"Altura: "<<(puntero+i)->altura<<endl;
                        cout<<"Peso: "<<(puntero+i)->peso<<endl;
                        cout<<"\n";
        }
}
```

Trabajo Práctico Nº 6

Tema: Estructuras de datos dinámicas: Pilas y Colas.

Objetivos:

* Entender las diferencias funcionales entre los arreglos y las estructuras dinámicas.
* Aprender a crear nodos en C.
* Aprender a crear estructuras de tipo pila, y dominar las operaciones básicas con ellas: Insertar nodo, borrar nodo, y recorrer la estructura completa para mostrar su información o realizar algún procesamiento con los datos almacenados en ella.
* Comprender la diferencia funcional entre la pila y la cola

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

PILAS:

Ejercicio Nº 1:

```
#include <stdio.h>
#include <malloc.h>
#define TAM 5
struct pila{
        int num;
        struct pila *sig;
};

int main(int argc, char **argv)
{
        int cont=0,i=1;
        struct pila *p, *base;
        base=NULL;
        /*p=malloc(sizeof(struct pila));
        printf("Ingrese numero:\n");
        scanf("%d",&p->num);
        p->sig=base;
        base=p;
        */
        while(cont<TAM)
        {       p=malloc(sizeof(struct pila));
                printf("Ingrese numero:\n");
                scanf("%d",&p->num);
                p->sig=base;
                base=p;
                cont++;
        }
        p=base;
        while(p!=NULL)
        {printf("Numero[%d]: %d\n",i++,p->num);
        p=p->sig;
        }
        return 0;
}
```

Ejercicio Nº 2:

```
#include <stdlib.h>
#include <stdio.h>

struct pila {
  int dato;
  struct pila *sig;
};


int main (void)
{
        struct pila *primero,*nuevo,*actual,*temp;
        int valor, eleccion=0;
        primero=NULL; // ORIGINALMENTE NO HAY NINGUN NODO
        printf("\n 1-Carga datos \n 2-Imprime pila \n 3-Borrar nodo\n 4-Salir\n");
        scanf("%d",&eleccion);
while(eleccion!=4)
{
        switch(eleccion)
        {
     case 1:
         nuevo = new(pila);
         if(nuevo!=NULL) //VERIFICAMOS QUE HAY MEMORIA DISPONIBLE
          {
                printf("Ingrese un valor a la pila\n\n");
                scanf("%d",&valor);
                (*nuevo).dato=valor;
                (*nuevo).sig=primero;
                primero=nuevo;
              }
              break;

         case 2:
          actual=primero;
          while(actual!=NULL)
           {
                    printf(" %d ",(*actual).dato);
                    actual=(*actual).sig;
              }
               break;
```

```
            case 3:

                    if(primero==NULL)
                     printf("La lista esta vacia\n\n");
                      else
                       {
                                        temp=primero;
                                                primero=(*primero).sig;
                                                free(temp);
                                        }

                     break;

                   case 4:
                    eleccion = 4;
                     break;
          }
    printf("\n 1-Carga datos \n 2-Imprime lista \n 3-Borrar datos\n  4-Salir\n");
          scanf("%d",&eleccion);
}
return(0);
}
```

Ejercicio Nº 3:

```
#include <stdio.h>
#include <malloc.h>


struct nodos{
        char ap[50];
        int edad;
        struct nodos *sig;
};

int main(int argc, char **argv)
{
        struct nodos *p,*base;
        base=NULL;
        while(p!=NULL){
                p=malloc(sizeof(struct nodos));
                printf("Ingrese Apellido:\n");
                scanf("%s",&p->ap[0]);
```

```
                printf("Ingrese edad:\n");
                scanf("%d",&p->edad);
                p->sig=base;
                base=p;
                if(p->edad==999)
                break;
        }
        p=base;
        printf("Apellidos de mayores de 21 años:\n");
        while(p!=NULL)
        {       if(((p->edad)>21)&&((p->edad)!=999))
                {printf("- %s\n",p->ap);
                printf("Edad: %d\n",p->edad);}
                p=p->sig;
        }
        return 0;
}
```

Ejercicio Nº 4:

```
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>
struct nodo
{       int num;
        struct nodo *sig;
};
int main(int argc, char **argv)
{       int count=0;
        char op;
        typedef struct nodo nodo;
        nodo *p,*base;

        base=NULL;
        do
        {       p=malloc(sizeof(nodo));
                if(!p)
                {printf("no hay memoria\n");
                exit (1);
                }
                printf ("ingrese el numero\n");
                scanf("%d",&p->num);
```

```
                p->sig=base;
                base=p;
                printf("desea ingresar otro nodo? 'S' o 'N'\n");
                scanf("%s",&op);
                count++;
                }
        while(op=='s'||op=='S');


        p=(base);

        while(p!=NULL)
        {       printf("numero:%d\n",(*(p-(count))).num);
                p=p->sig;
        }
        return 0;
}
```

COLAS:

Ejercicio Nº 1:

```
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>

struct nodo{
        char ap[50];
        int edad;
        struct nodo *sig;
};

int main(int argc, char **argv)
{       int bandera=0;
        struct nodo *pri, *ult, *p;
        pri=ult=NULL;
        do
        {
                p=malloc(sizeof(struct nodo));
                if(!p)
                {printf("No hay memoria\n");
                exit(1);
                }
```

```
                else
                {       printf("Ingrese apellido:\n");
                        scanf("%s",&p->ap[0]);
                        printf("Ingrese edad:\n");
                        scanf("%d",&p->edad);
                        p->sig=NULL;
                        if (pri==NULL)
                        {
                                pri=p;
                        }
                        else
                        {
                                ult->sig=p;
                        }
                        ult=p;
                        if (p->edad==999)
                        {
                                bandera=1;
                        }
                }

        } while (bandera!=1);
        p=pri;
        while (p!=NULL)
        {
                if ((p->edad>=21)&&(p->edad!=999))
                {
                        printf("-%s\t",p->ap);
                        printf("%d\n",p->edad);
                }
                p=p->sig;
        }


        return 0;
}
```

Ejercicio Nº 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <string.h>
struct cola
```

```
{       char ap[50];
        char sex;
        int edad;
        struct cola *sigc;
};
struct pila
{       char ap[50];
        char sex;
        int edad;
        struct pila *sigp;
};
int main(int argc, char **argv)
{       struct cola *pri,*ult,*pc;
        struct pila *base,*pp;
        int bandera=0;
        pri=NULL;
        ult=NULL;
        base=NULL;
        do
        {       pc=malloc(sizeof(struct cola));
                if(!pc)
                {       printf("no hay memoria\n");
                        exit(0);
                }
                else
                {       printf("ingrese apellido:\n");
                        scanf("%s",&pc->ap[0]);
                        printf("ingrese sexo, 'M' o 'F':\n");
                        scanf("%s",&pc->sex);
                        printf("ingrese edad:\n");
                        scanf("%d",&pc->edad);
                        pc->sigc=NULL;
                }
                if(pri==NULL)
                {pri=pc;}
                else
                {ult->sigc=pc;}
                ult=pc;
                if((pc->edad)==999)
                bandera=1;
        }while(bandera!=1);
        pc=pri;
        while(pc!=NULL)
```

```
	{	pp=malloc(sizeof(struct pila));
		if(!pp)
		{	printf("no hay memoria\n");
			exit (0);
		}
		else
			{
					if((pc->edad)>=18&&(pc->sex)=='F')
						if((pc->edad)!=999)
					{	strcpy(pp->ap,pc->ap);
						pp->edad=pc->edad;
						pp->sigp=NULL;
						base=pp;
					}

			}pc=pc->sigc;
	}
	pp=base;
	while(pp!=NULL)
	{	printf("apellido:%s\n",pp->ap);
		printf("edad:%d\n",pp->edad);
		pp=pp->sigp;
	}

	return 0;
}
```

Ejercicio Nº 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

struct nodo{
	int num;
	struct nodo *sig;
};

int main(int argc, char **argv)
{	char op;
	int cont=0,i=0;
	struct nodo *ult,*pri,*p;
```

```
	ult=NULL;
	pri=NULL;
	do
	{	p=malloc(sizeof(struct nodo));
		if(!p)
		{printf("No hay memoria.\n");
		exit(0);}
		else
		{	printf("Ingrese numero:\n");
			scanf("%d",&p->num);
			p->sig=NULL;
			if (pri==NULL)
			{	pri=p;}
			else
			{	ult->sig=p;}
			ult=p;}
		printf("Para ingresar otro nodo presione S o s:\n");
		scanf("%s",&op);
		cont++;
	} while (op=='S'||op=='s');
	p=pri;
	do
	{	printf("Número: %d\n",p->num);
		p=p->sig;
		i++;
	} while (i<(cont/2));

	return 0;
}
```

Ejercicio Nº 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <string.h>
struct alumnos{
	char ap[20];
	float prom;
	int ina;
	struct alumnos *sig;
};
```

```
int menu(void);
void promay(struct alumnos*p);
void inamenor(struct alumnos*p);
void promocion(struct alumnos*p);

int main(int argc, char **argv)
{       char op;
        struct alumnos *primero, *ultimo, *p;
        ultimo=NULL;
        primero=NULL;

        do
        {p=malloc(sizeof(struct alumnos));
        if(!p)
        {printf("No hay memoria.\n");
        exit(0);}
        else
        {       printf("Ingrese apellido:\n");
                scanf("%s",&p->ap[0]);
                printf("Ingrese promedio gral del alumno:\n");
                scanf("%f",&p->prom);
                printf("Ingrese cantidad de inasistencias:\n");
                scanf("%d",&p->ina);
                p->sig=NULL;
                if(primero==NULL)
                {primero=p;}
                else
                {ultimo->sig=p;}
                ultimo=p;
        }
        printf("Desea agregar otro alumno?: presione S o s.\n");
        scanf("%s",&op);
        }while(op=='S'||op=='s');
        p=primero;

        switch (menu())
        {case 1:{promay(p);
                        break;}
        case 2:{inamenor(p);
                        break;}
        case 3:{promocion(p);
                        break;}
```

```
        default:{break;}
        }
        return 0;
}

int menu(void)
{       int op=0;
        printf("Indique la opcion que desea:\n"
                        "1- Informacion del promedio mayor.\n"
                        "2- Información del alumno con menor cant de inasistencias.\n"
                        "3- Alumnos con promedio mayor a 7.\n"
                        "4- Salir.\n?:");
        scanf("%d",&op);
        return op;
}

void promay(struct alumnos*p)
{       float mayor=0;
        struct alumnos *res;
        res=malloc(sizeof(struct alumnos));
        while (p!=NULL)
        {       if (p->prom>=mayor)
        {       mayor=p->prom;
                strcpy(res->ap,p->ap);
                res->prom=p->prom;
                res->ina=p->ina;
        }
        p=p->sig;
        }
        printf("Apellido:%s\n",res->ap);
        printf("Promedio:%.2f\n",res->prom);
        printf("Inasistencias:%d\n",res->ina);
}

void inamenor(struct alumnos*p)
{       int menor;
        struct alumnos *res;
        res=malloc(sizeof(struct alumnos));
        menor=p->ina;
        while (p!=NULL)
        {       if ((p->ina)<=menor)
        {menor=p->ina;
                strcpy(res->ap,p->ap);
```

```
                res->prom=p->prom;
                res->ina=p->ina;
        }
        p=p->sig;
        }
        printf("Apellido:%s\n",res->ap);
        printf("Promedio:%.2f\n",res->prom);
        printf("Inasistencias:%d\n",res->ina);
}

void promocion(struct alumnos*p)
{       while(p!=NULL)
                { if((p->prom)>=7)
                        {               printf("Apellido:%s\n",p->ap);
                                        printf("Promedio:%.2f\n",p->prom);
                                        printf("Inasistencias:%d\n",p->ina);
                        } p=p->sig;
                }
}
```

Ejercicio Nº 5:

```
#include <stdlib.h>
#include <stdio.h>

struct pila {
  int dato;
  struct pila *sig;
};


int main (void)
{
        struct pila *primero,*nuevo,*actual,*temp;
        int valor, eleccion=0;
        primero=NULL; // ORIGINALMENTE NO HAY NINGUN NODO
        printf("\n 1-Carga datos \n 2-Imprime pila \n 3-Borrar nodo\n 4-Salir\n");
        scanf("%d",&eleccion);
while(eleccion!=4)
{
        switch(eleccion)
        {
     case 1:
```

```
        nuevo = new(pila);
        if(nuevo!=NULL) //VERIFICAMOS QUE HAY MEMORIA DISPONIBLE
         {
                printf("Ingrese un valor a la pila\n\n");
                scanf("%d",&valor);
                (*nuevo).dato=valor;
                (*nuevo).sig=primero;
                primero=nuevo;
              }
              break;

        case 2:
         actual=primero;
         while(actual!=NULL)
          {
                      printf(" %d ",(*actual).dato);
                      actual=(*actual).sig;
              }
                break;

          case 3:

                   if(primero==NULL)
                    printf("La lista esta vacia\n\n");
                     else
                      {
                                      temp=primero;
                                              primero=(*primero).sig;
                                              free(temp);
                                      }

                    break;

                  case 4:
                    eleccion = 4;
                    break;
        }
    printf("\n 1-Carga datos \n 2-Imprime lista \n 3-Borrar datos\n  4-Salir\n");
        scanf("%d",&eleccion);
}
return(0);
}
```

Trabajo Práctico Nº 7

Temas: Listas simples, organización de la estructura, creación de nodos, inserción de nodos, inserción ordenada, borrado de nodos, implementación de métodos para ingresar, recorrer mostrar y borrar información de la estructura. Repaso de puntero doble. Listas doblemente enlazadas, organización de la estructuras de datos, creación de nodos, inserción de nodos, inserción ordenada, borrado de nodos, implementación de métodos para insertar, recorrer en ambos sentidos, y mostrar, y borrar información de la estructura.

Objetivos:

* Entender la forma de enlazar los nodos en listas simples y doblemente enlazadas.
* Dominar las operaciones básicas con estas estructuras: Insertar nodo en forma ordenada, borrar nodo y recorrer la estructura completa.
* Comprender la diferencia entre la lista simple y doble.
* Aprender a diferenciar las listas de las estructuras pila y cola.
* Comprender las distintas aplicaciones importantes de las listas.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Listas Simples:

Ejercicio Nº 1:

```
#include <stdio.h>

#define N 10

struct nodo{
        float dato;
        struct nodo *ant;
        struct nodo *sig;
        };

int menu();
void may_men(struct nodo *);
void men_may(struct nodo *);
void imp(struct nodo *);


int main()
{struct nodo *primero,*ultimo,*p;
primero=ultimo=p=NULL;
        for(int i=0;i<N;i++){
                if((p=new(struct nodo))==NULL){
                printf("No hay memoria");}
                else{
                printf("Ingrese el dato %d:",i+1);
                scanf("%f",&p->dato);
                p->sig=NULL;
                p->ant=ultimo;
                if(primero==NULL)
                primero=p;
                else
                ultimo->sig=p;
                ultimo=p;}
}

p=primero;
int opcion=menu();
while(opcion!=3){
        switch(opcion){
        case 1:
```

```
                may_men(primero);
                imp(primero);
                break;
        case 2:
                men_may(primero);
                imp(primero);
                break;
}
opcion=menu();
}
printf("\n Fin del programa");
return 0;
}

int menu()
{ int opcion;
  printf(" Menu de opciones:\n"
        " 1-Ordenar de mayor a menor.\n"
        " 2-Ordenar de menor a mayor.\n"
        " 3-Salir\n");
         scanf("%d",&opcion);
         return opcion;
}

void may_men(struct nodo *primero)
{float aux;
 struct nodo *p,*q;
 int i,j;
 for(i=0;i<N;i++){
 p=primero;
        for(j=0;j<N-i-1;j++){
                q=p->sig;
                if(q->dato>p->dato){
                aux=q->dato;
                q->dato=p->dato;
                p->dato=aux;}
                p=q;}
  }
}

void men_may(struct nodo *primero)
{float aux;
struct nodo *p,*q;
```

```
int i,j;
        for(i=0;i<N;i++){
                p=primero;
                for(j=0;j<N-i-1;j++){
                        q=p->sig;
                        if(q->dato<p->dato){
                        aux=q->dato;
                        q->dato=p->dato;
                        p->dato=aux;
                        p=q;}
                }
 }
}

void imp(struct nodo *p){

printf("La lista es:\n");
        while(p!=NULL){
        printf("%.2f\n",p->dato);
        p=p->sig;}
}
```

ejercicio Nº 2:

```
#include <stdlib.h>
#include <stdio.h>

struct lista {
  char dato;
  struct lista *sig;
};

int main(int argc, char **argv)
{
        struct lista *p,*actual,*anterior,*primero;
        char dato;
        int i;
        p=actual=anterior=primero=NULL;
        for(i=0;i<5;i++)
        {
        p=new(lista);
        if(p!=NULL)
        {
```

```
                printf("Ingrese un caracter para agregar a la lista\n");
                scanf("%s",&dato);
                p->dato=dato;
                p->sig=NULL;
                if(primero==NULL)
                {
                        primero =p;
                }
        else
        {
                        if(dato<primero->dato)
                        {
                                p->sig=primero;
                                primero = p;
                  }
                  else
                  {
                                actual=primero;
                                while((actual!=NULL)&&(dato>actual->dato))
                                {
                                        anterior=actual;
                                        actual=actual->sig;
                                }
                           anterior->sig=p;
                           p->sig=actual;
                         }
                 }
}
}
actual=primero;
while(actual!=NULL)
{       static int i=1;
        printf("valor %d= %c\n ",i,actual->dato);
   actual=actual->sig;
   i++;
 }
}
```

Ejercicio Nº 3:

```
#include <stdlib.h>
#include <stdio.h>
using namespace std;
```

```
struct lista {
  char apellido[20];
  int edad;
  struct lista *sig;
};
int menu(void);
int main(int argc, char **argv)
{
        struct lista *p,*actual,*anterior,*primero;
        int bandera=0,op;
        p=actual=anterior=primero=NULL;
do{
        op=menu();
switch(op)
{
                case 1:
                        {       p=new(lista);
                                if(p!=NULL)
                                {       if(anterior!=NULL)
                                {anterior=actual;}
                                printf("ingrese apellido\n");
                                scanf("%s",p->apellido);
                                printf("Ingrese edad\n\n");
                                scanf("%d",&p->edad);
                                actual=p;
                                if(primero==NULL)
                                {primero=actual;}
                                p->sig=NULL;
                                }
                                break;
                        }
                case 2:
                        {       free(actual);
                                actual=anterior;
                                p=actual;
                                actual->sig=p;
                                p->sig=NULL;
                                break;
                        }

                case 3:
                {
                        actual=primero;
```

```
                        while(actual!=NULL)
                                {
                        printf("apellido: %s\n ",actual->apellido);
                        printf("edad= %d\n ",actual->edad);
                        actual=actual->sig;

                                }
                        break;}
                default:
                        {       printf("fin\n");
                                bandera=1;
                                break;}
}
}while(bandera==0);
        return 0;
}
int menu(void)
{       int opc;
        printf("\n1-guardar nodo\n2-borrar nodo\n3-mostrar nodos\n");
        scanf("%d",&opc);
        return opc;
}
```

Ejercicio Nº 4:

```
#include <stdio.h>
#include <malloc.h>
#include <string.h>

struct nodo{
      int edad;
      char ap[10];
      struct nodo *sig;
      };

int main()
{       struct nodo *p,*primero,*anterior;
        primero=anterior=p=NULL;
        int aux=1;
        while(aux==1){
                if((p=new(struct nodo))==NULL){
                        printf("No hay memoria.\n");}
                else{
```

```
                printf("Ingrese el apellido:\n ");
                scanf("%s",p->ap);
                printf("Ingrese edad (Para salir 999):\n ");
                scanf("%d",&p->edad);
        if(p->edad==999)
                aux=0;
        else{
                p->sig=NULL;
                //p->ant=ultimo;
        if(primero==NULL)
                primero=p;
        else
                anterior->sig=p;
                anterior=p;}
        }
}
p=primero;
printf("\nLos datos de las personas mayores de edad son:");
while(p!=NULL)
{       if((p->edad)>=21)
        printf("\nApellido:%s\nEdad:%d\n",p->ap,p->edad);
        p=p->sig;}
return 0;
}
```

Ejercicio Nº 5:

```
#include <stdlib.h>
#include <stdio.h>


struct lista {
  char dato;
  struct lista *sig;
};

int main(int argc, char **argv)
{
        struct lista *p,*actual,*anterior,*primero,*copia;
        char dato;
        int i;
        p=actual=anterior=primero=NULL;
        copia=NULL;
```

```
  for(i=0;i<5;i++)
  {
  p= new(lista);
  copia=new(lista);
  if(p!=NULL)
  {
          printf("Ingrese un caracter para agregar a la lista\n");
          scanf("%s",&dato);
          p->dato=dato;
          p->sig=NULL;
          copia->dato=dato;
          copia->sig=NULL;

          if(primero==NULL)
          {
                  primero =p;
          }
else
{
                  if(dato<primero->dato)
                  {
                          p->sig=primero;
                          primero = p;
             }
             else
             {
                          actual=primero;
                          while((actual!=NULL)&&(dato<actual->dato))
                          {
                                  anterior=actual;
                                  actual=actual->sig;
                          }
                    anterior->sig=p;
                    p->sig=actual;
                  }

                  if(dato>primero->dato)
                  {
                          copia->sig=primero;
                          primero = copia;
             }
             else
             {
```

```
                        actual=primero;
                        while((actual!=NULL)&&(dato<actual->dato))
                        {
                                anterior=actual;
                                actual=actual->sig;
                        }
                    anterior->sig=copia;
                    copia->sig=actual;
                }
            }
}
}
actual=primero;
while(actual!=NULL)
printf("Original:\n");
{       static int i=1;
        printf("valor %d= %c\n ",i,actual->dato);
   actual=actual->sig;
   i++;
 }
 actual=primero;
printf("Copia de la inversa:\n");
 while(actual!=NULL)
printf("Original:\n");
{       static int i=1;
        printf("valor %d= %c\n ",i,actual->dato);
   actual=actual->sig;
   i++;
 }
return 0;
}
```

Ejercicio Nº 6:

```
#include <stdlib.h>
#include <stdio.h>
#define N 5
struct lista {
        char nombre[20];
        int inasistencias;
        float prom;
        struct lista *sig;
};
```

```
int main(int argc, char **argv)
{
        struct lista *p,*actual,*anterior,*primero;
        int i;
        p=actual=anterior=primero=NULL;
        for(i=0;i<N;i++)
        {
        p=new(lista);
        if(p!=NULL)
        {       printf("Ingrese nombre\n\n");
                scanf("%s",p->nombre);
                printf("Ingrese promedio\n\n");
                scanf("%f",&p->prom);
                printf("Ingrese inasistencias\n\n");
                scanf("%d",&p->inasistencias);
                p->sig=NULL;
                if(primero==NULL)
                {
                        primero =p;
                }
      else
      {
                        if(p->inasistencias<primero->inasistencias)
                        {
                                p->sig=primero;
                                primero = p;
                  }
                  else
                  {
                                actual=primero;
                                while((actual!=NULL)&&(p->inasistencias>actual-
>inasistencias))
                                {
                                        anterior=actual;
                                        actual=actual->sig;
                                }
                     anterior->sig=p;
                     p->sig=actual;
                   }
                }
}
}
```

```
actual=primero;
while(actual!=NULL)
{
   printf("nombre= %s\n ",actual->nombre);
   printf("promedio= %.2f\n ",actual->prom);
        printf("inasistencias= %d\n ",actual->inasistencias);
   actual=actual->sig;
 }
}
```

Listas Dobles:

Ejercicio Nº 1:

Ejercicio Nº 6:

```
#include <stdlib.h>
#include <stdio.h>
#define N 5
struct lista {
        char nombre[20];
        int inasistencias;
        float prom;
        struct lista *sig;
};

int main(int argc, char **argv)
{
        struct lista *p,*actual,*anterior,*primero;
        int i;
        p=actual=anterior=primero=NULL;
        for(i=0;i<N;i++)
        {
        p=new(lista);
        if(p!=NULL)
        {       printf("Ingrese nombre\n\n");
                scanf("%s",p->nombre);
                printf("Ingrese promedio\n\n");
                scanf("%f",&p->prom);
                printf("Ingrese inasistencias\n\n");
                scanf("%d",&p->inasistencias);
                p->sig=NULL;
                if(primero==NULL)
```

```
            {
                primero =p;
            }
    else
    {
                if(p->inasistencias<primero->inasistencias)
                {
                    p->sig=primero;
                    primero = p;
              }
              else
              {
                    actual=primero;
                    while((actual!=NULL)&&(p->inasistencias>actual-
>inasistencias))
                    {
                        anterior=actual;
                        actual=actual->sig;
                    }
                  anterior->sig=p;
                  p->sig=actual;
               }
            }
}
}
actual=primero;
while(actual!=NULL)
{
   printf("nombre= %s\n ",actual->nombre);
   printf("promedio= %.2f\n ",actual->prom);
        printf("inasistencias= %d\n ",actual->inasistencias);
   actual=actual->sig;
 }
}
```

Ejercicio Nº 2:

```
#include <stdio.h>
#define N 10

struct nodo{
        char letra;
        struct nodo *sig;
        struct nodo *ant;
```

```
        };

void repetic(struct nodo *);

int main(){
struct nodo *primero,*ultimo,*p;
primero=ultimo=p=NULL;
        for(int i=0;i<N;i++){
        if((p=new(struct nodo))==NULL){
        printf("\n No hay memoria");}
        else{
                do{
                        printf("Ingrese letra %d:",i+1);
                        scanf("%s",&p->letra);}
                while((p->letra<65)&&(p->letra>90)&&(p->letra<97)&&(p->letra>122));
                p->sig=NULL;
                p->ant=ultimo;
                if(primero==NULL)
                primero=p;
                else
                ultimo->sig=p;
                ultimo=p;
                }
        }
repetic(primero);
printf("Fin del programa\n");
return 0;
}

void repetic(struct nodo *p)
{struct nodo *q;
printf("Las letras repetidas son:\n");
        while(p!=NULL){
        q=p->sig;
        if(p->letra!='0')
        while(q!=NULL){
                if(q->letra==p->letra)
                {printf("\n%c",p->letra);
                q->letra='0';}
                }
                p=p->sig;
      }
}
```

Ejercicio Nº 3:

```
#include <stdio.h>
#include <malloc.h>
#include <string.h>

struct nodo{
	int edad;
	char ap[10];
	struct nodo *sig;
	struct nodo *ant;
	};

int main()
{	struct nodo *p,*primero,*ultimo;
	primero=ultimo=p=NULL;
	int aux=1;
	while(aux==1){
		if((p=new(struct nodo))==NULL){
			printf("No hay memoria.\n");}
		else{
			printf("Ingrese el apellido:\n ");
			scanf("%s",p->ap);
			printf("Ingrese edad (Para salir 999): ");
			scanf("%d",&p->edad);
		if(p->edad==999)
			aux=0;
		else{
			p->sig=NULL;
			p->ant=ultimo;
		if(primero==NULL)
			primero=p;
		else
			ultimo->sig=p;
			ultimo=p;}
	 }
}
p=primero;
printf("Los datos de las personas mayores de edad son:");
while(p!=NULL)
{	if((p->edad)>=21)
	printf("Apellido:%s\nEdad:%d\n",p->ap,p->edad);
	p=p->sig;}
```

```
return 0;
}
```

Ejercicio Nº 4:

```
#include <stdio.h>

struct nodo{
          float dato;
          struct nodo *sig;
          struct nodo *ant;
          };

int main()
{         int tam,i;
          float *vect;
          struct nodo *p,*primero,*ultimo;
    p=primero=ultimo=NULL;
          printf("Ingrese tamaño del arreglo: ");
          scanf("%d",&tam);
          vect=new(float);
          for(i=0;i<tam;i++)
                    {printf(" Ingrese elemento %d: ",i+1);
                    scanf("%f",vect+i);}
          for(i=0;i<tam;i++){
                    if((p=new(struct nodo))==NULL){
                    printf("\nNo hay memoria.");}
                    else{
                    p->dato=*(vect+i);
                    p->sig=NULL;
                    p->ant=ultimo;
                    if(primero==NULL)
                    primero=p;
                    else
                    ultimo->sig=p;
                    ultimo=p;}
        }
          p=primero;
          while(p!=NULL){
          printf("Elemento: %.1f\n",p->dato);
          p=p->sig;}
          p=ultimo;
          printf("\n");
```

```
        while(p!=NULL){
        printf("Elemento: %.1f\n",p->dato);
        p=p->ant;}
return 0;
}
```

Ejercicio Anexo:

```
#include <stdlib.h>
#include <stdio.h>

struct lista {
  int dato;
  struct lista *sig;
};

int main(int argc, char **argv)
{
        struct lista *p,*actual,*anterior,*primero;
        int dato,i;
        p=actual=anterior=primero=NULL;
        for(i=0;i<5;i++)
        {
        p=new(lista);
        if(p!=NULL)
        {
                printf("Ingrese un dato para agregar a la lista\n\n");
                scanf("%d",&dato);
                p->dato=dato;
                p->sig=NULL;
                if(primero==NULL)
                {
                        primero =p;
                }
      else
      {
                        if(dato<primero->dato)
                        {
                                p->sig=primero;
                                primero = p;
                  }
                  else
                  {
```

```
                                actual=primero;
                                while((actual!=NULL)&&(dato>actual->dato))
                                {
                                        anterior=actual;
                                        actual=actual->sig;
                                }
                           anterior->sig=p;
                           p->sig=actual;
                        }
                }
}
}
actual=primero;
while(actual!=NULL)
{       int i=1;
        printf("valor %d= %d\n ",i,actual->dato);
   actual=actual->sig;
   i++;
 }
}
```

Ejercicio Anexo lista:

```
#include <iostream>
using namespace std;

struct lista
{
   int dato;
   struct lista *sig;
   struct lista *ant;
};


main ()
{
   struct lista *primero, *ultimo, *auxiliar;
   int i=0,rta,bandera=0;
   auxiliar = new (struct lista);

   cout<< "Ingrese dato 1: ";
   cin>>auxiliar->dato;
   auxiliar->sig=NULL;
   auxiliar->ant=NULL;
```

```
    primero = auxiliar;
    ultimo = auxiliar;

    while(bandera == 0)    //Carga
    {
       auxiliar = new (struct lista);
       cout<<"Ingrese el dato"<<i+2<<": ";
       cin>>auxiliar->dato;
       auxiliar->ant = ultimo;
       auxiliar->sig=NULL;

       ultimo->sig = auxiliar;
       ultimo = auxiliar;
       i++;
       cout<<"Presione 1 para seguir ingresando datos: ";
       cin>>rta;
       if (rta!=1)
       {bandera=1;}
    }

    //izquierda-derecha
    auxiliar=primero;
    cout<<"\n Izquierda-derecha: "<<endl;
    while( auxiliar!=NULL )
    {
       cout<<auxiliar->dato<<" - ";
       auxiliar = auxiliar->sig;
    }

    //derecha-izquierda
    auxiliar=ultimo;
    cout<<"\n derecha-izquierda: "<<endl;
    while( auxiliar!=NULL )
    {
       cout<<auxiliar->dato<<" - ";
       auxiliar = auxiliar->ant;
    }


}
```

Ejercicio Anexo:

```
struct nodo{
          float dato;
          struct nodo *ant;
          struct nodo *sig;
          };

int menu();
void may_men(struct nodo *);
void men_may(struct nodo *);
void imp(struct nodo *);


int main()
{struct nodo *primero,*ultimo,*p;
primero=ultimo=p=NULL;
        for(int i=0;i<N;i++){
                if((p=new(struct nodo))==NULL){
                printf("No hay memoria");}
                else{
                printf("Ingrese el dato %d:",i+1);
                scanf("%f",&p->dato);
                p->sig=NULL;
                p->ant=ultimo;
                if(primero==NULL)
                primero=p;
                else
                ultimo->sig=p;
                ultimo=p;}
}

p=primero;
int opcion=menu();
while(opcion!=3){
        switch(opcion){
        case 1:
                may_men(primero);
                imp(primero);
                break;
        case 2:
                men_may(primero);
                imp(primero);
                break;
}
```

```
opcion=menu();
}
printf("\n Fin del programa");
return 0;
}

int menu()
{ int opcion;
  printf(" Menu de opciones:\n"
        " 1-Ordenar de mayor a menor.\n"
        " 2-Ordenar de menor a mayor.\n"
        " 3-Salir\n");
         scanf("%d",&opcion);
         return opcion;
}

void may_men(struct nodo *primero)
{float aux;
 struct nodo *p,*q;
 int i,j;
 for(i=0;i<N;i++){
 p=primero;
         for(j=0;j<N-i-1;j++){
                  q=p->sig;
                  if(q->dato>p->dato){
                  aux=q->dato;
                  q->dato=p->dato;
                  p->dato=aux;}
                  p=q;}
  }
}

void men_may(struct nodo *primero)
{float aux;
struct nodo *p,*q;
int i,j;
         for(i=0;i<N;i++){
                  p=primero;
                  for(j=0;j<N-i-1;j++){
                           q=p->sig;
                           if(q->dato<p->dato){
                           aux=q->dato;
                           q->dato=p->dato;
```

```
                    p->dato=aux;
                    p=q;}
                }
 }
}

void imp(struct nodo *p){

printf("La lista es:\n");
        while(p!=NULL){
        printf("%.2f\n",p->dato);
        p=p->sig;}
}
```

Ejercicio Anexo:

```
#include <stdio.h>
#define N 10

struct nodo{
         char letra;
         struct nodo *sig;
         struct nodo *ant;
         };

void repetic(struct nodo *);

int main(){
struct nodo *primero,*ultimo,*p;
primero=ultimo=p=NULL;
        for(int i=0;i<N;i++){
        if((p=new(struct nodo))==NULL){
        printf("\n No hay memoria");}
        else{
                do{
                        printf("Ingrese letra %d:",i+1);
                        scanf("%s",&p->letra);}
                while((p->letra<65)&&(p->letra>90)&&(p->letra<97)&&(p->letra>122));
                p->sig=NULL;
                p->ant=ultimo;
                if(primero==NULL)
                primero=p;
                else
```

```
                ultimo->sig=p;
                ultimo=p;
                }
        }
repetic(primero);
printf("Fin del programa\n");
return 0;
}

void repetic(struct nodo *p)
{struct nodo *q;
printf("Las letras repetidas son:\n");
        while(p!=NULL){
        q=p->sig;
        if(p->letra!='0')
        while(q!=NULL){
                if(q->letra==p->letra)
                {printf("\n%c",p->letra);
                q->letra='0';}
                }
                p=p->sig;
      }
}
```

Ejercicio Anexo:

```
#include <stdio.h>
#include <malloc.h>
#include <string.h>

struct nodo{
      int edad;
      char ap[10];
      struct nodo *sig;
      struct nodo *ant;
      };

int main()
{       struct nodo *p,*primero,*ultimo;
        primero=ultimo=p=NULL;
        int aux=1;
        while(aux==1){
                if((p=new(struct nodo))==NULL){
```

```
                printf("No hay memoria.\n");}
            else{
                printf("Ingrese el apellido:\n ");
                scanf("%s",p->ap);
                printf("Ingrese edad (Para salir 999): ");
                scanf("%d",&p->edad);
            if(p->edad==999)
                aux=0;
            else{
                p->sig=NULL;
                p->ant=ultimo;
            if(primero==NULL)
                primero=p;
            else
                ultimo->sig=p;
                ultimo=p;}
        }
}
p=primero;
printf("Los datos de las personas mayores de edad son:");
while(p!=NULL)
{       if((p->edad)>=21)
        printf("Apellido:%s\nEdad:%d\n",p->ap,p->edad);
        p=p->sig;}
return 0;
}
```

Ejercicio Anexo:

```
#include <stdio.h>
struct nodo{
        float dato;
        struct nodo *sig;
        struct nodo *ant;
        };

int main()
{       int tam,i;
        float *vect;
        struct nodo *p,*primero,*ultimo;
   p=primero=ultimo=NULL;
        printf("Ingrese tamaño del arreglo: ");
```

```
        scanf("%d",&tam);
        vect=new(float);
        for(i=0;i<tam;i++)
                {printf(" Ingrese elemento %d: ",i+1);
                scanf("%f",vect+i);}
        for(i=0;i<tam;i++){
                if((p=new(struct nodo))==NULL){
                printf("\nNo hay memoria.");}
                else{
                p->dato=*(vect+i);
                p->sig=NULL;
                p->ant=ultimo;
                if(primero==NULL)
                primero=p;
                else
                ultimo->sig=p;
                ultimo=p;}
      }
        p=primero;
        while(p!=NULL){
        printf("Elemento: %.1f\n",p->dato);
        p=p->sig;}
        p=ultimo;
        printf("\n");
        while(p!=NULL){
        printf("Elemento: %.1f\n",p->dato);
        p=p->ant;}
return 0;
}
```

Ejercicio Anexo:

```
#include <stdlib.h>
#include <stdio.h>

struct lista {
  int dato;
  struct lista *sig;
};

int main (void){
        struct lista *p,*actual,*anterior,*primero;
        int dato,i;
```

```
        p=actual=anterior=primero=NULL;
        for(i=0;i<5;i++)
        {
        p=new(lista);
        if(p!=NULL)
        {
                scanf("Ingrese un dato para agregar a la lista\n\n");
                scanf("%d",&dato);
                p->dato=dato;
                p->sig=NULL;
                if(primero==NULL)
                {
                        primero =p;
                }
      else
      {
                        if(dato<primero->dato)
                        {
                                p->sig=primero;
                                primero = p;
                  }
                  else
                  {
                                actual=primero;
                                while((actual!=NULL)&&(dato>actual->dato))
                                {
                                        anterior=actual;
                                        actual=actual->sig;
                                }
                           anterior->sig=p;
                           p->sig=actual;
                         }
                 }
}
}
actual=primero;
while(actual!=NULL)
{
        printf(" %d ",actual->dato);
   actual=actual->sig;
 }
}
```

Ejercicio Anexo:

```
#include <stdlib.h>
#include <stdio.h>
struct lista {
  int dato;
  struct lista *sig;
  struct lista *anterior;
};

typedef struct lista * ptro;

void cargar(ptro *,ptro *);
void recorrer(ptro);
void recorrer2(ptro);
void borrar(ptro *,ptro *);
int main (void)
{
        ptro primero,ultimo;
        int eleccion=0;
        primero=ultimo=NULL; // ORIGINALMENTE NO HAY NINGUN NODO
        printf("\n 1-Carga datos \n 2-Imprime lista para una lado \n 3-Imprime lista para el
otro\n 4-Borra nodo\n, 5-Quit\n");
        scanf("%d",&eleccion);
while(eleccion!=5)
{
        switch(eleccion)
        {
     case 1:
          cargar(&primero,&ultimo);
                    break;

          case 2: printf("Recorrido para un lado\n\n");
                recorrer(primero);
                     break;
         case 3:  printf("Recorrido para el otro lado\n\n");
                     recorrer2(ultimo);
                          break;
         case 4:   borrar(&primero,&ultimo);
                      break;
         case 5:
```

```
                eleccion=5;
                break;
 }//FIN DE SWHITCH
   printf("\n 1-Carga datos \n 2-Imprime lista para una lado \n 3-Imprime lista para el
otro\n 4-Borra nodo\n, 5-Quit\n");
        scanf("%d",&eleccion);

}//FIN DE WHILE

return(0);
}

void cargar (ptro *primero, ptro *ultimo)
{
        int valor;
        ptro actual,nuevo,ant;
        actual=ant=nuevo=NULL;
        nuevo=new(lista);
        printf("Ingrese el valor\n");
        scanf("%d",&valor);
        (*nuevo).dato=valor;
        (*nuevo).sig=NULL;
        (*nuevo).anterior=NULL;
        if(*primero==NULL) //NO HAY NODOS, POR LO TANTO VA AL PRINCIPIO
         {
                (*nuevo).anterior=NULL;
                (*nuevo).sig=NULL;
                (*nuevo).dato=valor;
                *ultimo=*primero=nuevo;
         }
        else
         {      if(valor<(*primero)->dato) // EL NUEVO NODO VA AL PRINCIPIO
                {
                        (*primero)->anterior=nuevo;
                        (*nuevo).sig=*primero;
                        (*nuevo).anterior=NULL;
                        *primero=nuevo;
                }
                else
                {
                        if(valor>(*ultimo)->dato) //EL NODO VA AL FINAL DE LA LISTA
                        {
                          (*ultimo)->sig=nuevo;
```

```
                        (*nuevo).anterior=*ultimo;
                        (*nuevo).sig=NULL;
                        *ultimo=nuevo;
                    }
                    else //EL NODO VA AL MEDIO
                        {
                        actual=*primero;
                        while(valor>(*actual).dato)
                        {
                          ant=actual;
                          actual=(*actual).sig;
                         }
                          (*ant).sig=nuevo;
                                  (*nuevo).anterior=ant;
                          (*nuevo).sig=actual;
                          (*actual).anterior=nuevo;
          }
            }
}//FIN DE IF DE PRIMERO=NULL
}


void recorrer(ptro primero)
{
        ptro actual,anterior;
        actual=anterior=NULL;
        actual=primero;
        if(primero==NULL)
        printf("Lista vacia\n\n");
        else
        {
                while(actual!=NULL)
                {
                 printf("%d ",(*actual).dato);
                 actual=(*actual).sig;
                }
        }
}

void recorrer2(ptro ultimo)
{
        ptro actual,anterior;
        actual=anterior=NULL;
```

```
	actual=ultimo;
	if(ultimo==NULL)
	printf("Lista vacia\n\n");
	else
	{
		while(actual!=NULL)
		{
		 printf("%d ",(*actual).dato);
		 actual=(*actual).anterior;
		}
	}
}

void borrar(ptro *primero,ptro *ultimo)
{
	ptro temp,actual,anterior;
	int flag=0,dato;
	temp=actual=anterior=NULL;
if(*primero!=NULL)
{
	printf("Ingrese el dato a borrar\n\n");
	scanf("%d",&dato);
	if(dato==(*primero)->dato)
	{
		temp=*primero;
		*primero=(*primero)->sig;
		(*primero)->anterior=NULL;
		free(temp);
	}
	else
	{
		if(dato==(*ultimo)->dato)
		{
		  temp=*ultimo;
		  *ultimo=(*ultimo)->anterior;
		  (*ultimo)->sig=NULL;
	    }
	    else
	    {
		  actual=*primero;
		  while((dato!=(*actual).dato)&&(actual!=NULL))
		  {
			anterior=actual;
```

```
                    actual=(actual)->sig;

                }
                  if(actual!=NULL)
                  {
                        temp=actual;
                        actual=(actual)->sig;
                        (anterior)->sig=actual;
                        (actual)->anterior=anterior;
                        free(temp);
                  }
                   else
                   printf("El valor no esta en la lista!!");
            }
        }
}
else
printf("Lista vacia!!\n\n");
}
```

Ejercicio Anexo:

```
#include <stdio.h>

struct lista {
  int dato;
  struct lista *sig;
};


int main (void)
{
        struct lista *primero,*nuevo,*actual,*anterior,*temp;
        int valor, eleccion=0;
        primero=NULL; // ORIGINALMENTE NO HAY NINGUN NODO
        printf("\n 1-Carga datos \n 2-Imprime lista \n 3-Borrar datos\n 4-Salir\n");
        scanf("%d",&eleccion);
while(eleccion!=4)
{
        switch(eleccion)
        {
     case 1:
         nuevo = new(lista);
         if(nuevo!=NULL) //VERIFICAMOS QUE HAY MEMORIA DISPONIBLE
```

```
        {
                printf("Ingrese un valor a la lista\n\n");
                scanf("%d",&valor);
                (*nuevo).dato=valor;
                (*nuevo).sig=NULL;
                anterior=NULL;
                actual=primero;
                while((actual!=NULL)&&((*nuevo).dato>(*actual).dato)) //BUSCAMOS LA
POSICION DEL NUEVO NODO
                {
                        anterior=actual;
                        actual=(*actual).sig;
                }
                if(anterior==NULL) //ESTE ES EL CASO EN QUE NUNCA ENTRO AL
WHILE (INSERTA AL PRINCIPIO DE LA LISTA)
                {
                    (*nuevo).sig=primero;
                    primero=nuevo;
                }
                else //EN ESTE CASO EL NODO IRA ENTRE LOS PUNTEROS ANTERIOR
Y ACTUAL
                {     (*anterior).sig=nuevo;
                        (*nuevo).sig=actual;
                }
               }
               else
               {
                        printf("No hay memoria libre");
           }
              break;

          case 2:
           actual=primero;
           while(actual!=NULL)
           {
                    printf(" %d ",(*actual).dato);
                    actual=(*actual).sig;
              }
               break;

           case 3:

                if(primero==NULL)
```

```
                printf("La lista esta vacia\n\n");
                 else
                 {
                                printf("Ingrese el valor a borrar\n\n");
                                scanf("%d",&valor);
                                if(valor==(*primero).dato)
                                {
                                        temp=primero;
                                        primero=(*primero).sig;
                                        free(temp);
                                }
                                else
                                 {
                                        anterior=primero;
                                        actual=(*primero).sig;
                                        while((actual!=NULL)&&((*actual).dato!=valor))
                                        {
                                                anterior=actual;
                                                actual=(*actual).sig;
                                              }

                                        if(actual!=NULL)
                                         {
                                                        temp=actual;
                                                        (*anterior).sig=(*actual).sig;
                                                        free(temp);
                                              }
                                              else
                                              printf("\nEl valor no esta en la lista\n\n");
                                }

                                }
                 break;

               case 4:
                 eleccion = 4;
                  break;
        }
   printf("\n 1-Carga datos \n 2-Imprime lista \n 3-Borrar datos\n  4-Salir\n");
        scanf("%d",&eleccion);
}
return(0);
}
```

Trabajo Práctico Nº 8

Temas: Arboles binarios de búsqueda.

Objetivos:

* Comprender la definición de árbol en forma general como organización jerárquica no lineal de datos.
* Diferenciar los distintos conceptos relacionados a los arboles.
* Comprender la definición de árbol binario.
* Discriminar la terminología de los arboles binarios.
* Aprender a crear arboles binarios y recorrerlos.
* Diferenciar los recorridos pre-orden, in-orden y post-orden.
* Crear métodos en C para generar arboles y recorrerlos.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Ejercicio Nº 1:

```
#include <stdlib.h>
#include <stdio.h>

struct arbol {
  int dato;
  struct arbol *der;
  struct arbol *izq;
};

void insertar(struct arbol **, int);
void inorden(struct arbol *puntero);
void preorden(struct arbol *puntero);
void postorden(struct arbol *puntero);


int main (void)
{
        struct arbol *raiz;
        int i,valor;
        raiz=NULL;
        for(i=0;i<9;i++)
        {
                printf("Inserte un valor\n\n");
                scanf("%d",&valor);
                insertar(&raiz,valor);
   }

printf("\nRecorrido INORDEN\n\n");
inorden(raiz);
printf("\nRecorrido PREORDEN\n\n");
preorden(raiz);
printf("\nRecorrido POSTORDEN\n\n");
postorden(raiz);
return(0);
}

void insertar(struct arbol **p, int dato)
{
```

```
        if(*p==NULL)
         {
                // printf("entra al si\n\n");
                 *p=new(struct arbol);
                                (**p).dato=dato;
                                (**p).izq=NULL;
                                (**p).der=NULL;
         }
         else
         {
          if(dato<(**p).dato)
           insertar(&(**p).izq,dato);
           else
           insertar(&(**p).der,dato);
         }
}

void inorden(struct arbol *puntero)
{

        if(puntero!=NULL)
        {
    inorden((*puntero).izq);
    printf(" %d ",(*puntero).dato);
    inorden((*puntero).der);
  }
}


void postorden(struct arbol *puntero)
{

        if(puntero!=NULL)
        {
    postorden((*puntero).izq);
    postorden((*puntero).der);
    printf(" %d ",(*puntero).dato);
  }
}

void preorden(struct arbol *puntero)
{
```

```
        if(puntero!=NULL)
        {
                printf(" %d ",(*puntero).dato);
    preorden((*puntero).izq);
    preorden((*puntero).der);
 }
}
```

Ejercicio Nº 1.1:

```
#include <iostream>
#include<stdlib.h>
using namespace std;

struct arbol
{
   int dato;
   struct arbol *izq;
   struct arbol *der;
};

void carga(struct arbol**, int);
void preorden(struct arbol*);

int main()
{
   int i, vec[10];
   struct arbol* bonsai;
   bonsai= NULL;

   cout<< "Ingrese 10 numeros:\n";
   for(i=0;i<10;i++)
   {
      cout<< "dato "<<i+1<<": ";
      cin>> vec[i];
      carga(&bonsai, vec[i]);
   }

   cout<<"\n\nDatos ingresados: ";
   for(i=0;i<10;i++) cout<<vec[i]<<"-";

   cout<<"\n\nDatos en preorden segun arbol: ";
   preorden(bonsai);
```

```
    cout<< "\n\n\n";

    return 0;
}


void carga(struct arbol **bonsai_rec, int num)
{
    if (*bonsai_rec == NULL)
    {
        *bonsai_rec= new(struct arbol);

        (*bonsai_rec)->dato = num;
        (*bonsai_rec)->izq = NULL;
        (*bonsai_rec)->der = NULL;
    }

    else
    {
        if(num < (*bonsai_rec)->dato)
            carga ( &( (*bonsai_rec)->izq), num);

        else
            carga (&( (*bonsai_rec)->der), num);
    }
}


void preorden(struct arbol* bonsai_rec)
{
    if( bonsai_rec != NULL )
    {
        cout<< bonsai_rec->dato <<"-";
        preorden(bonsai_rec->izq);
        preorden(bonsai_rec->der);
    }
}
```

Ejercicio Nº 2:

```
#include<iostream>
#include<stdio.h>
#include<stdlib.h>
```

```
using namespace std;


struct arbol {
  int dato;
  struct arbol *der;
  struct arbol *izq;
};

typedef struct arbol * ptro;

void cargar(ptro *,int);
void inorden(ptro);
void preorden(ptro);
void postorden(ptro);

int main(void)
{
	ptro raiz;
	raiz=NULL;
	int dato,i;
	for(i=0;i<10;i++)
	{
		cout<<"Ingrese el dato "<<i+1<<"\n";
		cin>>dato;
		cargar(&raiz,dato);
	}
	cout<<"\n CON TYPEDEF";
	cout<<"\n Recorrido INORDEN";
	inorden(raiz);
	cout<<"\n Recorrido PREORDEN";
	preorden(raiz);
	cout<<"\n Recorrido POSTORDEN";
	postorden(raiz);
}

void cargar(ptro *tree,int dato)
{
	if(*tree==NULL)
	{
		*tree=new(arbol);
		(*tree)->dato=dato;
		(*tree)->izq=NULL;
```

```
                (*tree)->der=NULL;
        }
        else
        {
                if(dato>(*tree)->dato)
                cargar(&(*tree)->der,dato);
                if(dato<(*tree)->dato)
                cargar(&(*tree)->izq,dato);
        }
}

void inorden(ptro tree)
{
        if(tree!=NULL)
        {
                inorden(tree->izq);
                printf(" %d ",(tree)->dato);
                inorden(tree->der);
        }
}

void preorden(ptro tree)
{
        if(tree!=NULL)
        {
                printf(" %d ",(tree)->dato);
                preorden(tree->izq);
                preorden(tree->der);
        }
}

void postorden(ptro tree)
{
        if(tree!=NULL)
        {
                postorden(tree->izq);
                postorden(tree->der);
                printf(" %d ",(tree)->dato);
        }
}
```

Ejercicio Nº 3:

```
#include <iostream>
using namespace std;

struct nodo{
        int dato;
        struct nodo *iz;
        struct nodo *der;
        };

void cargar(int,struct nodo **);
void repetic(struct nodo *,struct nodo **,int);
void imp(struct nodo *);

int main()
{struct nodo *raiz,*copia;
raiz=copia=NULL;
int dato=1;
cout<<" Ingrese dato(Presione 0 para salir): ";
cin>>dato;
while(dato!=0){
        cargar(dato,&raiz);
        cout<<" Ingrese dato(Presione 0 para salir): ";
        cin>>dato;}
repetic(raiz,&copia,0);
cout<<"Arbol original\n";
imp(raiz);
cout<<"\nCopia del arbol:\n";
imp(copia);
return 0;
}

void cargar(int dato,struct nodo **raiz)
{struct nodo *p,*aux,*actual;
int dir;
```

```
p=new(struct nodo);
p->dato=dato;
p->iz=p->der=NULL;
if(*raiz==NULL)
        *raiz=p;
else{
        aux=*raiz;
        while(aux!=NULL){
        actual=aux;
        if((actual->dato)>=(p->dato)){
        dir=0;
        aux=actual->iz;}
        else{
        dir=1;
        aux=actual->der;}
        }
        if(dir==0)
        actual->iz=p;
        else
        actual->der=p;
}
}

void repetic(struct nodo *p,struct nodo **copia,int dato)
{if(p->iz!=NULL)
        repetic(p->iz,copia,p->dato);
if(p->dato!=dato)
        cargar(p->dato,copia);
if(p->der!=NULL)
        repetic(p->der,copia,p->dato);
}

void imp(struct nodo *p)
{if(p->iz!=NULL)
        imp(p->iz);
        cout<<"-"<<p->dato;
if(p->der!=NULL)
        imp(p->der);
}
```

Ejercicio Nº 3.3:

```
#include <iostream>
# define tam 100
using namespace std;

struct arbol
{
   struct arbol*der;
   struct arbol*izq;
   int dato;
};

void carga(struct arbol**, int);
void mostrar( struct arbol* );

int main ()
{
   struct arbol *planta;
   int valor[tam],i,num_elem;

   cout<<"Ingrese la cantidad de elementos que desea ingresar: ";
   cin>>num_elem;

   planta=NULL;  //Primer pasada
   for(i=0;i<num_elem;i++)
   {
      cout<<"Ingrese el dato "<<i+1<<endl;
      cin>>valor[i];
   }

   for(i=0;i<num_elem;i++)
   {
                carga(&planta,valor[i]);
       }

   cout<<endl;
   mostrar(planta);


   return 0;
}
```

```
void carga(struct arbol** arb_recibido, int numero)
{
   if ( *arb_recibido==NULL)  //Solo la primera vez
   {
      *arb_recibido=new(struct arbol);
      (*arb_recibido)->der = NULL;
      (*arb_recibido)->izq = NULL;
      (*arb_recibido)->dato= numero;
   }
   else
   {
      if( (*arb_recibido)->dato <  numero )
      {
         carga(  &(*arb_recibido)->izq, numero );
      }
      else
      {
         carga( &(*arb_recibido)->der, numero);
      }
   }
}


void mostrar ( struct arbol* arbol_rec )
{
   if (arbol_rec != NULL)
   {
      mostrar( arbol_rec->izq );
      mostrar( arbol_rec->der );
      cout<<(arbol_rec->dato)<<endl;
   }
}
```

Ejercicio Nº 4:

```
#include <stdio.h>
#include <malloc.h>

struct arbol {
int dato;
```

```
struct arbol *der;
struct arbol *izq;
};

void insertar(struct arbol **, int);
void compara(struct arbol **,struct arbol **,struct arbol **);
void inorden(struct arbol *puntero);

int main(int argc, char **argv)
{       struct arbol *arbol1;
        struct arbol *arbol2;
        struct arbol *raiz;
        raiz=arbol1=arbol2=NULL;
        int valor,i;
        printf("Ingrese el primer arbol:\n");
        for (i = 0; i < 4; i++)
                {printf("Inserte un valor\n\n");
                scanf("%d",&valor);
                insertar(&arbol1,valor);
                }
        printf("Ingrese el segundo arbol:\n");
        for (i = 0; i < 4; i++)
                {printf("Inserte un valor\n\n");
                scanf("%d",&valor);
                insertar(&arbol2,valor);
                }
        for (i = 0; i < 8; i++)

        {
        compara(&arbol1, &arbol2, &raiz);
}

        printf("\nArbol generado:\n");
        inorden(raiz);

        return 0;
}

void insertar(struct arbol **p, int dato)
{       if(*p==NULL)
        {
        *p=malloc(sizeof(struct arbol));
        (**p).dato=dato;
```

```
        (**p).izq=NULL;
        (**p).der=NULL;
        }
        else
        {
        if(dato<(**p).dato)
        insertar(&(**p).izq,dato);
        else
        insertar(&(**p).der,dato);
        }
}

void compara(struct arbol **p1, struct arbol **p2,struct arbol **pr)
{
        if ((**p1).dato>(**p2).dato)
        {insertar(pr,(**p2).dato);
        //(*p2)=((**p2).izq);
        //compara(p1,p2,pr);
        }
        else
        {insertar(pr,(**p1).dato);
        //(*p1)=((**p1).der);
        //compara(p1,p2,pr);
        }
}

void inorden(struct arbol *puntero)
{if(puntero!=NULL)
        {
        inorden((*puntero).izq);
        printf(" %d ",(*puntero).dato);
        inorden((*puntero).der);
        }
}
```

Ejercicio Nº 4.2:

```
#include <iostream>

using namespace std;

struct arb     ol
{
   struct arbol*der;
```

```
    struct arbol*izq;
    int dato;
};

struct pila
{
    int dato;
    struct pila *psig;
};

void carga(struct arbol**, int);
void mostrar( struct arbol* );

int main ()
{
        struct pila *duracel,*ultimo;
    struct arbol *planta;
    int valor[4],i,vec_pila[4]={1,2,3,4};

    duracel = new (struct pila);
    duracel->dato = vec_pila[0];
    duracel->psig = NULL;
    ultimo=duracel;

    for(i=0;i<3;i++)
    {
                ultimo = new(struct pila);
                ultimo->dato = vec_pila[i+1];
                ultimo->psig = duracel;
                duracel=ultimo;
        }

    for(i=0;i<4;i++)
    {
                valor[i] = ultimo->dato;
                carga(&planta,valor[i]);
                ultimo = ultimo->psig;
        }

    cout<<endl;
    mostrar(planta);
```

```
    return 0;
}



void carga(struct arbol** arb_recibido, int numero)
{
    if ( *arb_recibido==NULL)  //Solo la primera vez
    {
        *arb_recibido=new(struct arbol);
        (*arb_recibido)->der = NULL;
        (*arb_recibido)->izq = NULL;
        (*arb_recibido)->dato= numero;
    }
    else
    {
        if( (*arb_recibido)->dato <  numero )
        {
            carga(  &(*arb_recibido)->izq, numero );
        }
        else
        {
            carga( &(*arb_recibido)->der, numero);
        }
    }
}


void mostrar ( struct arbol* arbol_rec )
{
    if (arbol_rec != NULL)
    {
        mostrar( arbol_rec->izq );
        mostrar( arbol_rec->der );
        cout<<(arbol_rec->dato)<<endl;
    }
}
```

Ejercicio Nº 5:

```
#include <stdio.h>
#include <malloc.h>
```

```
#include <stdlib.h>

struct arbol
{int dato;
struct arbol *izq;
struct arbol *der;
};

void insertar(struct arbol **,int );
void inorden(struct arbol *);

int main(int argc, char **argv)
{
        int i,valor;
        struct arbol *raiz;
        raiz=NULL;
        printf("Ingrese datos del arbol binario:\n");
        for (i = 0; i < 4; i++)
        {
                printf("Ingrese dato:\n");
                scanf("%d",&valor);
                insertar(&raiz,valor);
        }
        printf("Arbol ingresado, in orden:\n");
        inorden(raiz);

        return 0;
}

void insertar(struct arbol **p, int dato)
{
        if (*p==NULL)
        {
                *p=malloc(sizeof(struct arbol));  //Creo el nodo
                (**p).dato=dato;              //Inicializo el nodo
                (**p).izq=NULL;
                (**p).der=NULL;
        }
        else
        {if ((**p).dato>dato)                         //
                {insertar(&(**p).izq,dato);}
                else
                {insertar(&(**p).der,dato);}
```

```
        }
}

void inorden(struct arbol *punt)          //Acomodo los datos para mostrar
{if(punt!=NULL)
        {inorden((*punt).izq);
        printf(" %d ",(*punt).dato);
        inorden((*punt).der);
        }
}
```

Ejercicio Nº 9:

```
#include <iostream>
using namespace std;

struct arbol
{
        int dato;
        struct arbol *der;
        struct arbol *izq;
};

void carga (struct arbol**,int );
void nodo(struct arbol**,struct arbol *);
void mostrar (struct arbol *);


int main ()
{
        struct arbol *nodo1,*nodo2,*resultado;
        int valor,i;

        nodo1 = nodo2 = resultado = NULL;


        cout<<"Ingrese el arobl 1: \n";
        for(i=0;i<4;i++)
        {
                cout<<"Ingrese el dato: ";
                cin>>valor;
```

```
            carga( &nodo1, valor);
        }

        cout<<"\n\nIngrese el arobl 2: \n";
        for(i=0;i<4;i++)
        {
            cout<<"Ingrese el dato: ";
            cin>>valor;
            carga(&nodo2,valor);
        }

        cout<<"\n\nTEMINO LA CARGA de los dos primeros arboles\n\n";

        nodo(&resultado, nodo1);
        nodo(&resultado, nodo2);

        cout<<"\n\nTEMINO LA CARGA del arbol final \n\n";

        mostrar( resultado);

        return 0;
}


void carga( struct arbol** p,int dat )
{
        if(*p==NULL)
        {
            *p = new (struct arbol);
            (*p)->dato=dat;
            (*p)->izq=NULL;
            (*p)->der=NULL;
        }
        else
        {
            if( (*p)->dato > dat )
            {carga( &(*p)->izq, dat);}
            else
            {carga( &(*p)->der, dat );}
        }
}
```

```
void nodo (struct arbol ** resul ,struct arbol* p)
{
        if( p!= NULL )
        {
                 nodo(  &(*resul), p->izq );
                 nodo(  &(*resul), p->der );
                carga( resul, p->dato );
        }
}


void mostrar (struct arbol * a)
{
        if(a!=NULL)
        {
                mostrar( a->izq );
                mostrar( a->der );
                cout<< a->dato;
        }
}
```

Ejercicio Nº 12:

```
#include <iostream>
# define tam 100
using namespace std;

struct arbol
{
   struct arbol*der;
   struct arbol*izq;
   int edad,faltas,id;
   char nombre,sexo;
   float prom;

};

void carga(struct arbol**,char,int, int, int, char, float);
void mostrar( struct arbol* );

int main ()
```

```
{
   struct arbol *planta;
   int i,num_elem,edad,faltas,id;
   char nombre,sexo;
   float prom;

   cout<<"Ingrese la cantidad de alumnos que desea ingresar: ";
   cin>>num_elem;

   planta=NULL;  //Primer pasada

   for(i=0;i<num_elem;i++)
   {
            cout<<"Ingrese el nombre del alumno: "<<i+1<<endl;
      cin>>nombre;
            cout<<"Ingrese la edad del alumno: "<<i+1<<endl;
      cin>>edad;
      cout<<"Ingrese la cantidad de inasistencias del alumno: "<<i+1<<endl;
      cin>>faltas;
      cout<<"Ingrese el ID del alumno: "<<i+1<<endl;
      cin>>id;
      cout<<"Ingrese el genero (m o f) del alumno: "<<i+1<<endl;
      cin>>sexo;
      cout<<"Ingrese el promedio del alumno: "<<i+1<<endl;
      cin>>prom;

            carga(&planta,nombre,edad,faltas,id,sexo,prom);
        }

   cout<<endl;
   mostrar(planta);


   return 0;
}



void carga(struct arbol** arb_recibido,char nombre_rec,int edad_rec,int faltas_rec,int
id_rec,char sexo_rec, float prom_rec)
{
   if ( *arb_recibido==NULL)  //Solo la primera vez
   {
```

```
        *arb_recibido=new(struct arbol);
        (*arb_recibido)->der = NULL;
        (*arb_recibido)->izq = NULL;
        (*arb_recibido)->nombre = nombre_rec;
        (*arb_recibido)->edad = edad_rec;
        (*arb_recibido)->faltas = faltas_rec;
        (*arb_recibido)->id = id_rec;
        (*arb_recibido)->sexo = sexo_rec;
        (*arb_recibido)->prom= prom_rec;
    }
    else
    {
        if( (*arb_recibido)->edad <  edad_rec )
        {
            carga(  &(*arb_recibido)->izq, nombre_rec, edad_rec, faltas_rec, id_rec, sexo_rec,
prom_rec );
        }
        else
        {
            carga( &(*arb_recibido)->der, nombre_rec, edad_rec, faltas_rec, id_rec, sexo_rec,
prom_rec);
        }
    }
}


void mostrar ( struct arbol* arbol_rec )
{
    if (arbol_rec != NULL)
    {
        mostrar( arbol_rec->izq );
        mostrar( arbol_rec->der );

        if( arbol_rec->prom > 7 )
        {
                        cout<<"Nombre"<<(arbol_rec->nombre)<<endl;
                        cout<<"Edad"<<(arbol_rec->edad)<<endl;
                        cout<<"Sexo"<<(arbol_rec->sexo)<<endl;
                        cout<<"inasistencias"<<(arbol_rec->faltas)<<endl;
                        cout<<"ID"<<(arbol_rec->id)<<endl;
                        cout<<"Promedio"<<(arbol_rec->prom)<<endl;
                }
                else
```

```
		{
			if(arbol_rec->faltas < 5)
			{
				cout<<"Nombre: "<<(arbol_rec->nombre)<<endl;
				cout<<"Edad: "<<(arbol_rec->edad)<<endl;
				cout<<"Sexo: "<<(arbol_rec->sexo)<<endl;
				cout<<"inasistencias: "<<(arbol_rec->faltas)<<endl;
				cout<<"ID: "<<(arbol_rec->id)<<endl;
				cout<<"Promedio: "<<(arbol_rec->prom)<<endl;
			}

		}
	}
}
```

Trabajo Práctico Nº 9

Temas: Programación orientada a objetos. (POO)

Objetivos:

* Comprender los conceptos básicos del paradigma orientado a objetos.
*Diferenciar el paradigma orientado a objeto del paradigma procedimental.
*Comprender la definición de componente.
*Discriminar los atributos de un objeto de sus procedimientos.
*Entender la abstracción de datos.
*Comprender y dominar la representación de clases.
*Aprender a crear objetos a partir de clases.
*Resolver problemas mediante el uso del paradigma orientado a objetos.
*Comprender la funcionalidad de los métodos: constructor y destructor.
*Aprender a usar funciones amigas.
*Comprender los conceptos básicos de herencia simple y múltiple.
*Comprender los conceptos de polimorfismo.
*Aprender a utilizar la sintaxis de C++ para herencia simple y múltiple.
*Reconocer clases abstractas y sus propósitos.
*Entender sobrecarga de operadores.
*Aprender a utilizar y reconocer la sintaxis de la sobrecarga de operadores.

Docentes: - Teórico: Profesor Dr. Eduardo Destefanis.
-Practico: Profesora Ing. Silvia Carrera.

Ejercicio Nº 1:

```
#include <iostream>
#include <math.h>
using namespace std;

class punto{
        private:
        int a, b;
        public:
        punto(int a1, int b1);
        void ingresa(int a1, int b1);
        //void muestra(int &a1, int &b1);
        float calculo(int a1, int b1);
};

punto::punto(int a1, int b1)
{       a=a1;
        b=b1;
}
void punto::ingresa(int a1, int b1)
{       a=a1;
        b=b1;
}

/*void punto::muestra(int &a1, int &b1)
{       a1=a;
        b1=b;
}*/

float punto::calculo(int a1, int b1)
{       float res;
        //res=sqrt(pow(a,2)+pow(b,2));
        return res=hypot(a,b);
}

int main(int argc, char **argv)
{       punto pun1(0,0);
        int x, y;
        float hyp=0;
        cout<< "Ingrese el valor de x"<<endl;
```

```
	cin>>x;
	cout<< "Ingrese el valor de y"<<endl;
	cin>>y;
	pun1.ingresa(x,y);
	hyp=pun1.calculo(x,y);
	cout<<"La hipotenusa es: "<<hyp<<endl;
	cout<<x  <<y<<endl;
	return 0;
}
```

Ejercicio Nº 2:

```
#include <iostream>
using namespace std;

class vector{
	private:
	int v[10];
	public:
	vector(int v1);
	void carga(int v1,int a);
	void maymen(void);
};

vector::vector(int v1)
{for (int i = 0; i < 10; i++)
{	v[i]=v1;
}
}

void vector::carga(int v1, int a)
{	v[a]=v1;
}

void vector::maymen()
{	int may, men;
	may=v[0];
	men=v[0];
	for (int i = 0; i < 10; i++)
	{
		if (v[i]>may)
		{	may=v[i];
```

```
                }
                if (v[i]<men)
                {       men=v[i];
                }
        }
        cout<<"El mayor es: "<<may<<endl;
        cout<<"El menor es: "<<men<<endl;
}

int main(int argc, char **argv)
{
        int x;
        static int b=0;
        vector vec1(0);

        for (int i = 0; i < 10; i++)
        {       cout<<"Ingrese el valor"<<i+1<<endl;
                cin>>x;
                vec1.carga(x,b);
                b++;
        }
        vec1.maymen();
        return 0;
}
```

Ejercicio Nº 3:

```
#include <iostream>
using namespace std;

class vector
{private:
                int vec[10];
public:
                vector();
                void carga(int,int);
                void ordena();
                void muestra(void);
};

vector::vector()
{for (int i = 0; i < 10; i++)
        {
```

```
        vec[i]=0;
        }
}

void vector :: carga(int a,int j)
{
        vec[j]=a;
}

void vector :: ordena(void)
{
        int bandera=0,aux=0,i=0;
        for ( i = 0; i < 9; i++)
        {
                if (vec[i]>vec[i+1])
                {bandera=1;}
        }
        if (bandera==1)
                {       cout<<"El arreglo está desordenado, y se ordenará."<<endl;
                        for (i = 0; i < 10-1 ; i++)
                        {       for (int j = 0; j < (10-1); j++)
                                {       if(vec[j]>vec[j+1])
                                        {aux=vec[j];
                                        vec[j]=vec[j+1];
                                        vec[j+1]=aux;}
                                }
                        }
                }
        else
        {cout<<"El arreglo ha sido cargado en forma ordenada."<<endl;}
}

void vector :: muestra(void)
{
        cout<<"El arreglo es:"<<endl;
        for (int i = 0; i < 10; i++)
        {
                cout<<"Valor"<<i+1<<":"<<endl;
                cout<<vec[i]<<endl;
        }
}

int main(int argc, char **argv)
```

```
{
        int valor=0;
        class vector vec1;
        cout<<"Ingrese los datos:"<<endl;
        for (int k=0; k < 10; k++)
        {
                cout<<"Valor"<<k+1<<":"<<endl;
                cin>>valor;
                vec1.carga(valor,k);
        }
        cout<<"Se van a mostrar los datos cargados"<<endl;
        vec1.ordena();
        vec1.muestra();

        return 0;
}
```

Ejercicio Nº 4:

```
#include <iostream>
#define N 3
using namespace std;

class matriz{
          int mat[N][N];
          public:
               matriz(); //constructor
               void cargar();
               void print();
               float media();
          };

matriz::matriz(){
             int i,j;
             for(i=0;i<N;i++)
                        for(j=0;j<N;j++)
                                   mat[i][j]=0;
             cout<<"Constructor de Matriz. Todo bien por aca. -.-\n\n";//inicio toooda la
matriz a 0.
             }

void matriz::cargar(){
                int i,j;
```

```
                for(i=0;i<N;i++)
                        for(j=0;j<N;j++){
                                cout<<"M["<<i<<"]["<<j<<"]=";
                                cin>>mat[i][j];              //ingreso el valor.
                                }
                }

void matriz::print(){
                 int i,j;
                 for(i=0;i<N;i++){
                         // cout<<"\n";
                          for(j=0;j<N;j++)
                                  cout<<mat[i][j]<<" 2";  //muestro el valor.
                          }
                }

float matriz::media(){
                  float prom=0;
                  int i,j;
                  for(i=0;i<N;i++)
                           for(j=0;j<N;j++)
                                    prom+=mat[i][j];
                  return prom/(N*N);
                  }

int main(){
        class matriz arreglo;
        arreglo.cargar();
        arreglo.print();
        cout<<"\nEl promedio es: "<<arreglo.media();
        return 0;
        }
```

Ejercicio Nº 4:

```
#include <iostream>
#include <stdlib.h>
using namespace std;

struct nodo{
         int dato;
         struct nodo *sig;
```

```
        };

class pila{
        struct nodo *estructura,*base;
        public:
            pila(){base=NULL;}
            void insertar();
            void borrar();
            void prt();
        };

void pila::insertar(){
              estructura=new(struct nodo);
              cout<<"Ingrese el dato:";
              cin>>estructura->dato;
              estructura->sig=base;
              base=estructura;
              }

void pila::borrar(){
             if(base==NULL)
                        cout<<"\n No hay datos que borrar.";
             else{
                 estructura=base;
                 base=estructura->sig;
                 cout<<"\n Eliminando...";
                 delete(estructura);
                 }
             }

void pila::prt(){
            estructura=base;
            cout<<" Los datos son: ";
            while(estructura!=NULL){
                        cout<<estructura->dato<<"  ";
                        estructura=estructura->sig;
                        }
            }

int menu(){
        int op;
        cout<<"MENU:\n"
            "1 -> Ingresar Dato..\n"
```

```
            "2 ->Eliminar dato..\n"
            "3 ->Mostrar todo..\n"
            "4 ->SALIR.\n";
        cin>>op;
        return op;
        }

int main(){
        class pila objeto_pila;
        int op=1;
        while(op!=4){
                op=menu();
                switch(op){
                        case 1:
                              objeto_pila.insertar();
                              system("clear");
                              break;
                        case 2:
                              objeto_pila.borrar();
                               system("clear");
                              break;
                        case 3:
                              objeto_pila.prt();
                               system("clear");
                              break;
                        case 4:
                             cout<<"\n\n SALIENDO...";
                             break;
                        default:
                               cout<<"\n La opcion no pertenece al menu.";
                        }
                }
        return 0;
        }
```

Ejercicio Nº 6:

```
#include <iostream>
#include <stdlib.h>

using namespace std;
```

```
struct nodo{
          int dato;
          struct nodo *sig;
          };
class cola{
          struct nodo *estructura,*primero,*ultimo;
          public:
               cola(){estructura=primero=ultimo=NULL;}  //constructor: todos los punteros a
NULL.
               void inserta_nodo();
               void eliminar();
               void print();
          };

void cola::inserta_nodo(){
                  estructura=new(struct nodo);
                  cout<<"Dato a ingresar: ";
                  cin>>estructura->dato;
                  estructura->sig=NULL;
                  if(primero==NULL)
                                primero=estructura;
                  else
                     ultimo->sig=estructura;
                  ultimo=estructura;
                  }

void cola::eliminar(){
                 if(primero==NULL)
                                cout<<"Elemento 0: nada que borrar.";
                 else{
                     estructura=primero;
                     primero=primero->sig;
                     delete estructura;
                     cout<<"Correctamente eliminado!";
                     }
                 }

void cola::print(){
                  estructura=primero;
                  cout<<"Datos:\n";
                  while(estructura!=NULL){
                                cout<<"\n"<<estructura->dato;
```

```
                    estructura=estructura->sig;
                    }
            }

int menu(){
        system("clear");
        cout<<"Menu:\n"
            "1 ->Insertar dato.\n"
            "2 ->Borrar dato.\n"
            "3 ->Mostrar todo.\n"
            "4 ->exit.\n"
            "opcion:";
        int opcion;
        cin>>opcion;
        return opcion;
        }


int main(){
        class cola objeto_cola;
        int opcion=menu();
        while(opcion!=4){
                switch(opcion){
                        case 1:
                            objeto_cola.inserta_nodo();
                            break;
                        case 2:
                            objeto_cola.eliminar();

                            break;
                        case 3:
                            objeto_cola.print();
                            break;
                        default:
                             cout<<"\n\n La opcion ingresada no pertenece al menu.";

                        }
                opcion=menu();
                }
        return 0;
        }
```

Ejercicio Nº 7:

```
#include <iostream>

#define max_line 31

using namespace std;

class persona{
        char nombre[max_line];
        int edad;
        int sexo;
        long int dni;
        public:
            void cargar();
            int define_sexo();
        };

void persona::cargar(){
                cout<<"\n\n Ingrese el nombre: ";
                cin>>nombre;
                cout<<" Ingrese la edad: ";
                cin>>edad;
                cout<<" Ingrese el sexo: ";
                cin>>sexo;
                cout<<" Ingrese el DNI: ";
                cin>>dni;
                }
int persona :: define_sexo() {return sexo;};
int main(){
        class persona personas[5];
        int i,M=0,V=0;
        for(i=0;i<5;i++)
                    personas[i].cargar();
        for(i=0;i<5;i++){
                if(personas[i].define_sexo()==1)
                    V++;else M++;
                }
        cout<<"cantidad de Hombres: "<<V<<"cantidad de Mujeres: "<<M;
        return 0;
        }
```

Ejercicio Nº 8:

```
#include <iostream>
#include <stdlib.h>
#include <math.h>

using namespace std;

class alasar{
          int n;
          public:
               alasar(){n=rand();}
               int ver(){return n;}
               ~alasar(){cout<<"\n\n Se inicio el destructor.-. Ennemy Down!";}
          };

int main(){
         class alasar aleatorio;
         cout<<" El n es: "<<aleatorio.ver();
         return 0;
         }
```

Ejercicio Nº 9:

```
#include <iostream>
#include <stdlib.h>

using namespace std;

class alazar{
            unsigned int numero_aleatorio;
            public:
                 alazar();
                 int imprimir();
            };

alazar::alazar(){
                 int cota_sup,cota_inf, media;
                 srand(time(NULL));
                 do{
                   cout<<"Ingrese la cota superior: ";
                   cin>>cota_sup;
                   cout<<"Ingrese la cota inferior: ";
```

```
                cin>>cota_inf;
                }while(cota_inf>cota_sup);
                media = cota_sup-cota_inf;
              numero_aleatorio=rand()%(media);
              }

int alazar::imprimir(){return numero_aleatorio;}

int main(){
        class alazar numero;
        cout<<" El numero generado aleatoriamente es: "<<numero.imprimir();
        return 0;
        }
```

Ejercicio Nº 10:

```
#include <iostream>
#include <stdlib.h>


using namespace std;

class vector{
         int Vector[20];
         int lim;
         public:
              vector(); //constructor de la clase
              void ver_dato          ();
              ~vector(); //destructor de la clase.
         };

int comprobar(int *Vector,int i){
                    int aux;
                    for(aux=0;aux<i;aux++)
                    if(*(Vector+i)==*(Vector+aux))
                    return 1;
                    return 0;
                    }
vector::vector(){

            int rep,i;
            srand(time(NULL));
```

```
            lim=rand()%(20-1);
            for(i=0;i<lim;i++){
                        do{
                          Vector[i]=rand();
                          rep=comprobar(Vector,i);
                          }while(rep==1);
                        }
            }

vector::~vector(){delete[] Vector;}

void vector::ver_dato(){
                  cout<<"\n\nVector:  ";
                  for(int k=0;k<lim;k++)
                    cout<<Vector[k]<<" ";
                  }

int main(){
          class vector numero_aleatorio;
          numero_aleatorio.ver_dato();
          return 0;
          }
```

Ejercicio Nº 11:

```
#include <iostream>
using namespace std;

class vector{
           int Dim;
           int *V;
           public:
                 vector();
                 void cargar_vector();
                 void imprimir_elementos();
                 void buscar_mayor(int *,int *);
                 void repetidos();
           };

vector::vector(){
              do{
```

```
            cout<<"Dimension del vector: ";
            cin>>Dim;
            }while(Dim<1);
          V=new int(Dim);
          cout<<"Constructor: Iniciando en 0...\n\n";
          for(int i=0;i<Dim;i++){*(V+i)=0;}
                    }
void vector::cargar_vector(){
            for(int i=0;i<Dim;i++){
                        cout<<"V["<<i+1<<"]=";
                        cin>>*(V+i);
                        if(*(V+i)==0)
                                break;
                        }
            }
void vector::repetidos(){
          int j,i;
          int vectb[Dim];
      for(i=0;i<Dim;i++)
        vectb[i]=0;
      for(i=0;i<Dim;i++)
         if(*(V+i)!=0)
           for(j=i+1;j<Dim;j++)
                if(*(V+i)==*(V+j)){
                     vectb[i]++;
                     *(V+j)=0;
                                }
      for(i=0;i<Dim;i++)
        if(vectb[i]!=0) {cout<<"El elemento "<<*(V+i)<<" está "<<vectb[i]+1<<" veces.";}
                 }

void vector::imprimir_elementos(){
            cout<<"\nElementos: ";
            for(int i=0;i<Dim;i++)
            cout<<*(V+i)<<"\t\n";
            }

void vector::buscar_mayor(int *M,int *P)
{
       *M=*V;
       *P=1;
                        for(int i=1;i<Dim;i++)
          if(*(V+i)>*M)
```

```
                                        {
                    *M=*(V+i);
                    *P=i+1;
                }
}



int main(){
        class vector vector_objeto;
        int extremo,lugar;
        vector_objeto.cargar_vector();
        vector_objeto.imprimir_elementos();
        vector_objeto.buscar_mayor(&extremo,&lugar);
        cout<<"\nMAYOR:"<<extremo<<" HUBICACION:"<<lugar<<".\n";
        vector_objeto.repetidos();
        return 0;
        }
```

Ejercicio Nº 12:

```
#include <iostream>
#include <stdlib.h>

using namespace std;

class vector;
class matriz{
          int M[4][4];
          float prom;
          public:
                matriz();
                void cargar();
                void mostrar();
                void promedio();
                friend int comparar(class matriz,class vector);
          };
```

```
class vector{
        int V[10];
        float prom;
        public:
            vector();
            void cargar();
            void mostrar();
            void promedio();
            friend int comparar(matriz,vector);
        };


matriz::matriz(){
          int i,j;
          for(i=0;i<4;i++)
          for(j=0;j<4;j++)
             M[i][j]=0;
          }

vector::vector(){ for(int i=0;i<10;i++) V[i]=0;}

void matriz::cargar()
{
              int i,j;
              cout<<"\nIngrese la matriz:\n";
              for(i=0;i<4;i++)
              for(j=0;j<4;j++)
              {
                 cout<<"M["<<i+1<<"]["<<j+1<<"]=";
                 cin>>M[i][j];
              }
}

void vector::cargar()
{
              int i;
              cout<<"\n Ingrese el vector:\n";
              for(i=0;i<10;i++)
              {
              cout<<"V"<<i+1;
              cin>>V[i];
              }
}
```

```
void matriz::mostrar()
{
                    int i,j;
                    cout<<"\n matriz:";
                    for(i=0;i<4;i++)
                      {
                       cout<<"\n";
                    for(j=0;j<4;j++)
                       cout<<"\t"<<M[i][j];
                      }
}

void vector::mostrar()
{
                    int i;
                    cout<<"\n VECTOR:\n";
                    for(i=0;i<10;i++)
                       cout<<"\t"<<V[i];
}

void matriz::promedio()
{
                     prom=0;
                     int i,j;
                     for(i=0;i<4;i++)
                     for(j=0;j<4;j++)
                     prom+=M[i][j];
                     prom=prom/16;
}

void vector::promedio()
{
                     prom=0;
                     int i;
                     for(i=0;i<10;i++)
                     prom+=V[i];
                     prom=prom/10;
}

int comparar(matriz M,vector V)
{
                          if(M.prom>V.prom)
```

```
                    return 1;
                  else{
                  if(M.prom==V.prom)
                    return 0;
                  else
                    return -1;
                     }
}

int main(){
        class matriz objeto_matriz;
        class vector objeto_vector;
        objeto_matriz.cargar();
        objeto_vector.cargar();
        objeto_matriz.mostrar();
        objeto_vector.mostrar();
        objeto_matriz.promedio();
        objeto_vector.promedio();
        if(comparar(objeto_matriz, objeto_vector)==1)
                        cout<<"El prom de la matriz es mayor.";

        else{
            if(comparar(objeto_matriz, objeto_vector)==0)
                            cout<<"ambos promedios son iguales.";
            else
               cout<<"el promedio mayor es el del vecotor";
            }


        return 0;
        }
```

Ejercicio Nº 13:

```
#include <iostream>
using namespace std;

class punto
{       protected:
                int a,b;//centro de la circ
        public:
```

```
                punto (int a1, int b1) {a=a1;b=b1;};
                void ingresa (int a1, int b1);
};

class cir: public punto        //acceso publico?
{       private:
                int r;   //radio de la circ
        public:
                cir(int a, int b, int r1): punto(a,b) {r=r1;};
                void ingresac (int r1);
                void muestra();
};

void cir:: muestra ()
{       int a1,b1,r1;
                r1=r;
                a1=a;
                b1=b;
                cout<< "radio: "<<r1<<endl<< "x: "<<a1<<endl<< "y: "<<b1<<endl;
}
void punto :: ingresa (int a1, int b1)
{       a=a1;
        b=b1;
}
void cir :: ingresac (int r1)
{       r=r1;
}


int main(int argc, char **argv)
{int x,y,ra;
        punto pun1(0,0);
        cir circ1(0,0,0);
        cout<< "ingrese x"<<endl;
        cin>> x;
        cout<< "ingrese y"<<endl;
        cin>> y;
        pun1.ingresa(x,y);
        cout<< "ingrese radio"<<endl;
        cin>> ra;
        circ1.ingresac(ra);
        circ1.ingresa(x,y);
        circ1.muestra();
```

```
	return 0;
}
```

Ejercicio Nº 14:

```
#include <iostream>
#include <string.h>
#include <malloc.h>
using namespace std;

class persona
{	protected:
		char nom[20];
		char ape[20];
		char sex;
		int edad;
		char est[20];
	public:
		persona(char n[],char ap[], char s, int e, char ec[]);
};

class empleado:public persona
{	private:
	char categ[20];
	int ant;
	float rem;
	public:
		empleado(char [], char[], char, int ,char[], char[], int, float);
		void ingresa(char [],char [], char, int, char [], char[], int, float);
		void muestra(char [],char [], char , int , char [], char [], int , float);
};

persona :: persona(char n[],char ap[], char s, int e, char ec[])
{	strcpy(nom,n);
	strcpy(ape,ap);
	sex=s;
	edad=e;
	strcpy(est,ec);
}
empleado::empleado(char n[],char ap[], char s, int e, char ec[], char cat[], int antig, float
```

```
remu): persona(n,ap,s,e,ec)
{       strcpy(categ,cat);
        ant=antig;
        rem=remu;
}
void empleado:: ingresa(char n[],char ap[], char s, int e, char ec[], char cat[], int antig, float
remu)
{       strcpy(categ,cat);
        ant=antig;
        rem=remu;
}

void empleado:: muestra(char n[],char ap[], char s, int e, char ec[], char cat[], int antig,
float remu)
{cout<<n<<ap <<s <<e <<ec <<cat <<antig <<remu <<endl;
}

int menu (void);
void carga (int, empleado);

int menu (void)
{       int opc=0;
        cout<<"Ingrese la opción que desea realizar:"<<endl<<"1.Cargar
datos"<<endl<<"2.Mostrar datos"<<endl<<"3.Salir"<<endl;
        cin>>opc;
        return opc;
}
void carga(int contador, empleado emp1[])
{       char nombre[20],apellido[20],sexo,estado[20],catego[20];
        int eda, antigu;
        float remune;
        cout<< "Ingrese los datos del empleado:"<<endl;
        cout<< "ingrese nombre(letra):"<<endl;
        cin>>nombre;
        cout<< "ingrese apellido(letra):"<<endl;
        cin>>apellido;
        cout<< "ingrese sexo(letra):"<<endl;
        cin>>sexo;
        cout<< "ingrese edad:"<<endl;
        cin>>eda;
        cout<< "ingrese estado civil(letra):"<<endl;
        cin>>estado;
        cout<< "ingrese categoría(letra):"<<endl;
```

```
	cin>>catego;
	cout<< "ingrese antigüedad:"<<endl;
	cin>>antigu;
	cout<< "ingrese remuneración:"<<endl;
	cin>>remune;
	emp1[contador].ingresa(nombre,apellido,sexo,eda,estado,catego,antigu,remune);
}

int main(int argc, char **argv)
{	int opcion,bandera=0;
	static int cont=0;
	char nombre[20],apellido[20],sexo,estado[20],catego[20];
	int eda, antigu;
	float remune;

	class empleado *emp[5];
	for (int i = 0; i < 5; i++)
	{	emp[i] = new
empleado(nombre,apellido,sexo,eda,estado,catego,antigu,remune);
	}

	cout<< "\t Menu principal:"<<endl;

	while(bandera==0)
	{switch (opcion=menu())
	{	case 1:{carga(cont,emp[cont]);
				cont++;
				if(cont==5)
				{cout<<"Ya ingresó todos los datos posibles."<<endl;
				break;}
				break;}
		case 2:{if(cont==0)
				{cout<<"ERROR-No hay datos ingresados!"<<endl;
				break;}
					cout<<"Los datos ingresados son:"<<endl;
					for (int i = 0; i < cont; i++)
					{cout<<"Empleado "<<i+1<<":"<<endl;
					emp[i]-
>muestra(nombre,apellido,sexo,eda,estado,catego,antigu,remune);}
				break;}
		default:{cout<<"El programa ha finalizado."<<endl;
				bandera=1;
				break;}
```

```
        }

        }
        return 0;
}
```

Ejercicio Nº 15:

```
#include <iostream>
#include <stdlib.h>
using namespace std;

class coordenada
{       protected:
                int x;
                int y;
        public:
                coordenada(int a, int b){x=a;y=b;}
                void genera(int a, int b);
};

void coordenada::genera(int a, int b)
{a=rand()%100;
b=rand()%100;
cout<<a<<"   "<<b<<endl;}

class complejo: public coordenada
{       public:
        complejo(int a, int b):coordenada(a,b){x=a;y=b;}
        void muestra(void){
                genera(x,y);
                cout<<"Z="<<x<<"+j"<<y<<endl;}
};

int main(int argc, char **argv)
{       int op=0;
        complejo c(0,0);
        cout<<"Desea generar una complejo? 1 por si"<<endl;
        cin>>op;
        if (op==1)
        {
        c.muestra();}
        else
```

```
	{cout<<"El programa ha finalizado."<<endl;
	}

	return 0;
}
```

Ejercicio Nº 16:

```
#include <iostream>
#include <string.h>
#define tam 3

using namespace std;

int menu(void);

class persona{
	protected:
		char nombre[20];
		char apell[20];
		int edad;
		char sexo;
	public:
		persona();
		void cargap(char [],char [],int,char);
		friend void muestra(class persona, class alumno);
		friend void funb(class persona, class alumno);
};

class alumno:public persona{
	private:
		float prom;
		int inasis;
		int legajo;
	public:
		alumno(char nom[], char ap[], int eda, char sex);
		void cargaa(float,int,int);
		friend void muestra(class persona, class alumno);
		void funa();
		friend void funb(class persona, class alumno);
};
```

```
persona::persona()    // CONSTRUCTOR
{}

void persona :: cargap(char nom[], char ap[], int eda, char sex)
{
        strcpy(nombre,nom);
        strcpy(apell,ap);
        edad=eda;
        sexo=sex;
}

alumno::alumno(char nom[], char ap[], int eda, char sex):persona()   //CONSTRUCTOR
{}

void alumno :: cargaa(float prome, int ina, int leg)
{
        prom=prome;
        inasis=ina;
        legajo=leg;
}

void alumno :: funa()
{
        if (inasis>=3)
        {cout<<"-Legajo: "<<legajo<<endl;}
}

void funb(class persona p2, class alumno a2)
{
        if (a2.prom>=7)
                {cout<<"Alumno: "<<p2.apell<<endl;}
}

void muestra(class persona p1, class alumno a1)
{
        cout<<"Nombre: "<<p1.nombre<<endl;
        cout<<"Apellido: "<<p1.apell<<endl;
        cout<<"Edad: "<<p1.edad<<endl;
        cout<<"Sexo: "<<p1.sexo<<endl;
        cout<<"Promedio general: "<<a1.prom<<endl;
        cout<<"Inasistencias: "<<a1.inasis<<endl;
        cout<<"Legajo: "<<a1.legajo<<endl<<endl;
```

```
}


int main(int argc, char **argv)
{
	int bandera=0,cont=0,opcion=0,eda,leg,ina;
	char nom[20],ap[20],sex;
	float prome;

	class persona *per[tam];
	for (int i = 0; i < tam; i++)
		{per[i] = new persona();}

	class alumno *alum[tam];
	for (int i = 0; i < tam; i++)
		{alum[i] = new alumno(nom,ap,eda,sex);}

	while(bandera==0)
	{switch (opcion=menu())
	{	case 1:{cout<<"Ingrese los datos del alumno"<<endl;
				cout<<"Nombre:\t";
				cin>>nom;
				cout<<"Apellido:\t";
				cin>>ap;
				cout<<"Edad:\t";
				cin>>eda;
				cout<<"Sexo (M o F):\t";
				cin>>sex;
				cout<<"Promedio general:\t";
				cin>>prome;
				cout<<"Inasistencias:\t";
				cin>>ina;
				cout<<"Legajo:\t";
				cin>>leg;
				per[cont]->cargap(nom,ap,eda,sex);
				alum[cont]->cargaa(prome,ina,leg);
				cont++;
				if(cont==tam)
				{cout<<"Ya ingresó todos los datos posibles."<<endl;
				break;}

				break;}
```

```
		case 2:{if(cont==0)
				{cout<<"ERROR-No hay datos ingresados!"<<endl;
				break;}
					cout<<"Los datos ingresados son:"<<endl;
					for (int i = 0; i < cont; i++)
					{cout<<"Alumno "<<i+1<<":"<<endl;
					muestra(*per[i],*alum[i]);}
				break;}

		case 3:{if(cont==0)
				{cout<<"ERROR-No hay datos ingresados!"<<endl;
				break;}
				cout<<"Los legajos de los alumnos con mas de 3 inasistencias
son:"<<endl;
					for (int i = 0; i < cont; i++)
					{alum[i]->funa();}
				break;}

		case 4:{if(cont==0)
				{cout<<"ERROR-No hay datos ingresados!"<<endl;
				break;}
				cout<<"Alumnos sobresalientes:"<<endl;
				for (int i = 0; i < cont; i++)
					{funb(*per[i],*alum[i]);}
				break;}

		default:{cout<<"El programa ha finalizado."<<endl;
				bandera=1;
				break;}
	}
	}

	return 0;
}

int menu (void)
{	int opc=0;
	cout<<endl<<"---------->>>>Menu principal<<<<----------"<<endl;
	cout<<"Ingrese la opción que desea realizar:"<<endl<<"1.Cargar
datos"<<endl<<"2.Mostrar datos"<<endl;
	cout<<"3.Mostrar legajos con mas de 3 inasistencias"<<endl;
	cout<<"4.Apellidos de alumnos sobresalientes"<<endl;
	cout<<"5.Salir"<<endl<<"Opcion--->";
```

```
        cin>>opc;
        return opc;
}
```

Ejercicio Nº 18:

```
#include <iostream>
using namespace std;

class vehiculo
{       protected:
                int ano;
                int costo;
        public:
                vehiculo();     //constructor
                virtual void carga(int a, int b, int c);
                virtual void muestra(void);          // Siempre virtual en la BASE
};

class vehic_terr: public vehiculo
{       private:
                int kilom;
        public:
                vehic_terr(int a, int b);
                virtual void carga(int a, int b, int c){ano=a;costo=b;kilom=c;};
                virtual void muestra(void){cout<<ano<<costo<<kilom<<endl;};
        // Misma funcion sin virtual, estoy en la derivada

};

class vehic_aereo: public vehiculo
{       private:
                int horasv;
        public:
                vehic_aereo(int a, int b);
                virtual void carga(int a, int b, int c){ano=a;costo=b;horasv=c;};
                virtual void muestra(void){cout<<ano<<"\t"<<costo<<"\t"<<horasv<<endl;};
                        // Misma funcion sin virtual, estoy en la derivada
};

vehiculo::vehiculo()
{}
```

```
vehic_terr::vehic_terr(int a, int b):vehiculo()
{}

vehic_aereo::vehic_aereo(int a, int b):vehiculo()
{}

int main(int argc, char **argv)
{       int c=0,d=0,e, op=0;
        class vehiculo **v;
        v = new vehiculo*[3];
        v[0]= new vehiculo();
        v[1]= new vehic_terr(c,d);
        v[2]= new vehic_aereo(c,d);
        cout<<"Menu:"<<endl<<"Que desea ingresar?"<<endl;
        cout<<"1.Terrestre"<<endl<<"2.Aereo"<<endl;
        cin>>op;
        cout<<"ingrese año:"<<endl;
        cin>>c;
        cout<<"Ingrese costo:"<<endl;
        cin>>d;
        cout<<"Ingrese kilometraje u hora de vuelo:"<<endl;
        cin>>e;
        v[op]->carga(c,d,e);
        cout<<"Los datos ingresados son:"<<endl;
        cout<<"Año   Costo   Kilom/Hora"<<endl;
        v[op]->muestra();
        cout<<"Fin del programa"<<endl;
        return 0;
}
```

Ejercicio Nº 19:

```
#include <iostream>
#include <stdlib.h>
#include <math.h>
using namespace std;

class mesa{
        protected:
                int color;
                int material;
```

```
            int cant_patas;
        public:
            mesa();
            int mostrar_color(void);
            int mostrar_material(void);
            int mostrar_cant_patas(void);
        };



mesa::mesa(){
          cout<<" Ingrese el color: ";
          cin>>color;
          cout<<" Ingrese el tipo de material: ";
          cin>>material;
          cout<<" Ingrese el numero de patas: ";
          cin>>cant_patas;
          }

class cuadrado{
           protected:
                    float perimetro;
                    float area;
                    float lado;
           public:
                cuadrado();
                float mostrar_lado(void);
                float mostrar_perimetro(void);
                float mostrar_area(void);
           };

cuadrado::cuadrado(){
                cout<<" Ingrese el lado: ";
                cin>>lado;
                perimetro=lado*4;
                area=pow(lado,2);
                }

class circulo{
           protected:
                  float radio;
                  float perimetro;
                  float area;
```

```
        public:
            circulo();
            float mostrar_radio(void);
            float mostrar_perimetro(void);
            float mostrar_area(void);
        };


int mesa :: mostrar_color(void) {return color;};
int mesa :: mostrar_material (void) {return material;}
int mesa :: mostrar_cant_patas(void) {return cant_patas;}
float cuadrado :: mostrar_lado (void) {return lado;};
float cuadrado :: mostrar_perimetro(void){return perimetro;}
float cuadrado :: mostrar_area(void){return area;}
float circulo :: mostrar_radio (void) {return radio;};
float circulo :: mostrar_perimetro(void){return perimetro;}
float circulo :: mostrar_area(void){return area;}

circulo::circulo(){
            cout<<" Ingrese el radio: ";
            cin>>radio;
            perimetro=3.14*radio*2;
            area=3.1416*pow(radio,2);
            }

class MesaRedonda:public mesa,public circulo{
                        public:
                            MesaRedonda():mesa(),circulo(){}
                        };

class MesaCuadrada:public mesa,public cuadrado{
                        public:
                            MesaCuadrada():mesa(),cuadrado(){}
                        };

int main(){
        cout<<"mesa redonda:\n";
        class MesaRedonda objeto_mesa_redonda;
        cout<<"\nmesa cuadrada:\n";
        class MesaCuadrada objeto_mesa_cuadrada;
        cout<<" datos de mesa redonda son:\n";
        cout<<"·Color: "<<objeto_mesa_redonda.mesa::mostrar_color()<<"\n";
        cout<<"·Material: "<<objeto_mesa_redonda.mesa::mostrar_material()<<"\n";
```

```
        cout<<"·Patas: "<<objeto_mesa_redonda.mesa::mostrar_cant_patas()<<"\n";
        cout<<"·Radio: "<<objeto_mesa_redonda.circulo::mostrar_radio()<<"\n";
        cout<<"·Perimetro: "<<objeto_mesa_redonda.circulo::mostrar_perimetro()<<"\n";
        cout<<"·Area: "<<objeto_mesa_redonda.circulo::mostrar_area()<<"\n";

        cout<<"\ndatos mesa cuadrada son:\n";
        cout<<"·Color: "<< objeto_mesa_cuadrada.mesa::mostrar_color()<<"\n";
        cout<<"·Material: "<< objeto_mesa_cuadrada.mesa::mostrar_material()<<"\n";
        cout<<"·Patas: "<< objeto_mesa_cuadrada.mesa::mostrar_cant_patas()<<"\n";
        cout<<"·Lado: "<< objeto_mesa_cuadrada.cuadrado::mostrar_lado()<<"\n";
        cout<<"·Perimetro: "<<
objeto_mesa_cuadrada.cuadrado::mostrar_perimetro()<<"\n";
        cout<<"·Area: "<< objeto_mesa_cuadrada.cuadrado::mostrar_area()<<"\n";
        return 0;
        }
```

Ejercicio Nº 20:

```
#include <iostream>
#include <malloc.h>
using namespace std;

class vehiculo{
        protected:
                int ano;
                int costo;
        public:
                vehiculo();
                virtual int mostrar(){cout<<ano<<endl<<costo<<endl;return 0;};
};

class vehi_aereo: public vehiculo{
        private:
                int horasv;
        public:
                vehi_aereo(int ,int);
                void cargava(int a,int b,int c){ano=a;costo=b;horasv=c;};
                virtual int mostrar(){cout<<endl<<ano<<" "<<costo<<"
"<<horasv<<endl;return 0;};
};

vehiculo::vehiculo()
{}
```

```
vehi_aereo::vehi_aereo(int a,int b):vehiculo()
{}

int main(int argc, char **argv)
{
	int a=0,b=0,c=0,op=0,cont=0;

	/*class vehi_aereo* *v_a;
	v_a=new vehi_aereo*[2];
	v_a[0]=new vehi_aereo(a,b);
	v_a[1]=new vehi_aereo(a,b);*/



	class vehi_aereo *v_a[2];
	for (int i = 0; i < 2; i++)
		{v_a[i]=new vehi_aereo(a,b);}

	do{
	cout<<"Ingrese Año:"<<endl;
	cin>>a;
	cout<<"Ingrese costo:"<<endl;
	cin>>b;
	cout<<"Ingrese horas de vuelo:"<<endl;
	cin>>c;
	v_a[cont]->cargava(a,b,c);
	cont++;
	cout<<"Desea seguir ingresando? Presione 0"<<endl;
	cin>>op;
	}while ((op!=1)&&(cont<=2));
	cout<<endl<<"Mostramos datos guardados"<<endl;
	for (int i = 0; i < cont; i++)
	{
		v_a[i]->mostrar();
	}

	return 0;
```

-Sobrecarga de operadoes:

I)

```
#include <iostream>
#include <stdlib.h>
#include <string.h>
using namespace std;

void menor(int *);
void menor(int *,int *);

int main(void)
{

        int vect1[5]={0,2,-90,1,99};
        int vect2[5]={-1,2,90,1,99};
        menor(vect1);
   menor(vect2);
        menor(vect1,vect2);
}

void menor(int *p)
{
        int menor,i;
        menor=*p;
        for(i=0;i<5;i++)
        {
          if(*(p+i)<menor)
          menor=(*p+i);
        }
 cout<<"El menor elemento del arreglo es: "<<menor<<"\n\n";
}

void menor(int *p,int *q)
{
        int menor,i;
        menor=*p;
        for(i=0;i<5;i++)
        {
          if(*(p+i)<menor)
          menor=*(p+i);
        }

        for(i=0;i<5;i++)
        {
          if(menor<(*q+i))
```

```
            menor=(*q+i);
          }
 cout<<"El menor total es: "<<menor;
}
```

II)

```
#include<iostream>
#include<stdlib.h>
#include<string.h>
using namespace std;

int cuadrado(int);
float cuadrado(float);

int main(void)
{
        int x;
        float y;
        cout<<"Ingrese un entero\n\n";
        cin>>x;
        cout<<"\nIngrese un flotante\n\n";
        cin>>y;
        cout<<"\nEl cuadradado de "<<x<<"="<<cuadrado(x);
        cout<<"\nEl cuadradado de "<<y<<"="<<cuadrado(y);
}

int cuadrado(int x)
{
        return(x*x);
}

float cuadrado(float y)
{
        return(y*y);
}
```

III)

```
#include<iostream>
#include<stdlib.h>
#include<string.h>
using namespace std;

struct complejo
{
	int x;
	int y;
};

complejo operator + (complejo,complejo);

int main(void)
{
	struct complejo a,b,resultado;
	a.x=10;
	a.y=20;
	b.x=20;
	b.y=10;
	resultado=a+b;
	cout<<"La suma de los dos complejos es: "<<resultado.x<<"+J"<<resultado.y;
}

complejo operator + (complejo a, complejo b)
{
	complejo temp;
	temp.x=a.x+b.x;
	temp.y=a.y+b.y;
	return(temp);
}
```

IV)

```
#include<iostream>
#include<stdlib.h>
#include<string.h>
#include<math.h>
using namespace std;

class complejo
{
```

```
        private:
          int x;
          int y;
        public:
          complejo();
          char signo();
          void cargar();
          void mostrar();
          class complejo operator / (class complejo);
};



int main(void)
{
        class complejo a,b,resultado;
        a.cargar();
        b.cargar();
        resultado=a/b;
        resultado.mostrar();
}

complejo::complejo()
{
        x=0;
        y=0;
        cout<<"Objeto inicializado OK\n\n";
}

void complejo::cargar()
{
        cout<<"Ingrese la parte real\n\n";
        cin>>x;
        cout<<"Ingrese la parte imaginaria\n\n";
        cin>>y;
}

char complejo::signo()
{
        if(y>0)
        return('+');
        else
        return('-');
```

```
}

void complejo::mostrar()
{
        cout<<x<<signo()<<fabs(y);
   cout<<"\n\n";
}

complejo complejo::operator / (class complejo v)
{
        class complejo temp;
        temp.x=x+v.x;
        temp.y=y+v.y;
        return(temp);
}
```

V)

```
#include<iostream>
#include<stdlib.h>
#include<string.h>
#include<stdio.h>
#include<math.h>

using namespace std;

class complejo
{
        private:
          int x;
          int y;
        public:
          complejo();
          char signo();
          void cargar();
          void mostrar();
          float operator / (class complejo);
};



int main(void)
{
```

```
        class complejo a,b;
        float resultado;
        a.cargar();
        b.cargar();
        resultado=a/b;
        printf("\n Resultado: %f",resultado);
}

complejo::complejo()
{
        x=0;
        y=0;
        cout<<"Objeto inicializado OK\n\n";
}

void complejo::cargar()
{
        cout<<"Ingrese la parte real\n\n";
        cin>>x;
        cout<<"Ingrese la parte imaginaria\n\n";
        cin>>y;
}

char complejo::signo()
{
        if(y>0)
        return('+');
        else
        return('-');
}

void complejo::mostrar()
{
        cout<<x<<signo()<<fabs(y);
   cout<<"\n\n";
}

float complejo::operator / (class complejo v)
{
        class complejo temp;
        temp.x=x+v.x;
        temp.y=y+v.y;
        return(3.2);
```

```
}
```